REGEX

CHEAT SHEETS

A Complete Reference for Regular Expressions in Every Major Language

Character Classes · Quantifiers · Anchors · Groups & Backreferences · Lookaround · Flags · Replacement · Performance · Python · Java · JavaScript · Go · C# · Ruby · grep · sed · awk · Common Patterns

25 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

01 Introduction & Flavors
02 Literals, Metacharacters & Escaping
03 How the Engine Matches

CHARACTER CLASSES

04 Character Classes & Shorthands
05 POSIX Classes & Unicode

QUANTIFIERS & ANCHORS

06 Quantifiers
07 Greedy, Lazy & Possessive
08 Anchors & Word Boundaries

GROUPS, BACKREFERENCES & LOOKAROUND

09 Groups & Alternation
10 Named Groups & Backreferences
11 Lookahead & Lookbehind

FLAGS, REPLACEMENT & ADVANCED

12 Flags & Inline Modifiers
13 Substitution & Replacement Syntax
14 Advanced Constructs
15 Performance & Catastrophic Backtracking

PYTHON

16 Python re — Core Functions
17 Python — Substitution & the regex Module

JAVA

18 Java — Pattern & Matcher
19 Java — String Methods, Replacement & Gotchas

JAVASCRIPT

20 JavaScript — RegExp Basics
21 JavaScript — Replace & Modern Features

OTHER LANGUAGES & COMMAND-LINE TOOLS

22 Other Languages — Go, C#, Ruby & More
23 Command-Line Tools — grep, sed, awk & rg

QUICK REFERENCE — COMMON PATTERNS

24 Quick Reference — Validation Patterns
25 Quick Reference — Text Processing & Extraction

Covers PCRE2 · ECMAScript 2025 · Python 3.13 · Java · .NET · Go · grep / sed / awk (2026)

Begin → Sheet 01

REGEX

Table of Contents

25 sheets · syntax · examples · notes

FOUNDATIONS 01 – 03

01 Introduction & Flavors | 02 Literals, Metacharacter… | 03 How the Engine Matches

CHARACTER CLASSES 04 – 05

04 Character Classes & Sh… | 05 POSIX Classes & Unico…

QUANTIFIERS & ANCHORS 06 – 08

06 Quantifiers | 07 Greedy, Lazy & Possess… | 08 Anchors & Word Bound…

GROUPS, BACKREFERENCES & LOOKAROUND 09 – 11

09 Groups & Alternation | 10 Named Groups & Backr… | 11 Lookahead & Lookbehi…

FLAGS, REPLACEMENT & ADVANCED 12 – 15

12 Flags & Inline Modifiers | 13 Substitution & Replace… | 14 Advanced Constructs | 15 Performance & Catastr…

PYTHON 16 – 17

16 Python re — Core Func… | 17 Python — Substitution …

JAVA 18 – 19

18 Java — Pattern & Matc… | 19 Java — String Methods,…

JAVASCRIPT 20 – 21

20 JavaScript — RegExp B… | 21 JavaScript — Replace …

OTHER LANGUAGES & COMMAND-LINE TOOLS 22 – 23

22 Other Languages — Go,… | 23 Command-Line Tools …

QUICK REFERENCE — COMMON PATTERNS 24 – 25

24 Quick Reference — Vali… | 25 Quick Reference — Tex…

Introduction & Flavors

WHAT A REGEX IS

A **regular expression** (regex) is a compact pattern that describes a set of strings. You hand the pattern to an **engine**, and it scans text for substrings that fit: to validate input, find and extract data, split text, or search-and-replace. The core syntax (`.` `*` `+` `?` `[...]` `(...)` `|` `^` `$`) is nearly universal, but every language ships its own **flavor** with different extras, escaping rules and replacement syntax. Learn the core once, then learn your host's differences (sheets 16–23).

BACKGROUND

- 1950s Kleene defines regular languages; 1968 Thompson adds them to QED, then `ed` and `grep`
- 1987 Perl makes regex a core feature; 1997 **PCRE** brings Perl syntax to C
- Most modern flavors are Perl-style; POSIX tools (`grep` , `sed` , `awk`) keep an older dialect
- Test at **regex101.com** with the flavor set to your target language; it explains every token

FIRST PATTERNS (PYTHON)

```
import re
text = "Call 555-1234 or 555-9876 today"
re.search(r"\d{3}-\d{4}", text).group()    # '555-1234'
re.findall(r"\d{3}-\d{4}", text)           # ['555-1234', '555-9876']
re.sub(r"\d{3}-(\d{4})", r"XXX-\1", text)  # 'Call XXX-1234 or XXX-9876 today'
bool(re.fullmatch(r"[A-Z]{2}\d{4}", "AB1234"))  # True
bool(re.fullmatch(r"[A-Z]{2}\d{4}", "AB123"))   # False
# The same pattern in other hosts (note the escaping):
#   JavaScript  /\d{3}-\d{4}/g
#   Java        Pattern.compile("\\d{3}-\\d{4}")
#   C#          new Regex(@"\d{3}-\d{4}")
#   Go          regexp.MustCompile(`\d{3}-\d{4}`)
#   grep (ERE)  grep -Eo '[0-9]{3}-[0-9]{4}' calls.txt
```

CODE EXPLAINED

- → `\d{3}` — exactly three digits; `-` is a literal hyphen
- → `search` — first match anywhere; `findall` — every match
- → `sub` — replace; `(\d{4})` captures, `\1` reinserts it
- → `fullmatch` — the whole string must fit (validation)
- → `r"..."` / `@"..."` / backticks — raw strings keep `\` intact
- → POSIX ERE has no `\d` ; use `[0-9]` or `[[:digit:]]`

MAJOR FLAVORS

FLAVOR	USED BY	ENGINE	NOTABLE DIFFERENCES
PCRE2 / Perl	PHP `preg_*`, `grep -P`, R, nginx, Perl	backtracking	The reference feature set: recursion, verbs, \K, possessive
ECMAScript	JavaScript, TypeScript	backtracking	No possessive / atomic; lookbehind since ES2018; also C++ default
Java	`java.util.regex`, Kotlin, Scala	backtracking	Possessive, \p{…}, class intersection &&
.NET	C#, F#, PowerShell	backtracking	Variable-length lookbehind, balancing groups, opt-in linear mode
Python re	Python	backtracking	(?P<name>…); no \p{…} (use the `regex` module)
Ruby (Onigmo)	Ruby	backtracking	^ $ are always line anchors
RE2 / Rust	Go, Rust, ripgrep, BigQuery	automata	Linear time; no backreferences or lookaround
POSIX BRE / ERE	`grep`, `sed`, `awk`, `vi`	varies	Leftmost-longest; BRE needs \(\{; no \d \w

⚠ COMMON MISTAKES

- Testing in one flavor, shipping in another: a lookbehind that works in regex101's PCRE mode won't compile in Go
- Assuming `\d` and `\w` work everywhere: POSIX `grep -E` and `sed -E` need `[0-9]` or `[[:digit:]]`
- Using regex where a parser belongs (HTML, JSON, nested data); see sheet 25

✓ TIPS & BEST PRACTICES

- Set regex101's flavor to match your code before you start
- Use your host's raw-string syntax so backslashes reach the engine unchanged
- Test against strings that should **not** match, not just ones that should

QUICK SUMMARY

- `regex` — a pattern language for matching text
- `flavor` — host dialect; core syntax is shared
- `backtracking` vs `automata` — features vs guaranteed speed
- `regex101` — test with the right flavor selected

Literals, Metacharacters & Escaping

LITERAL VS SPECIAL

Most characters in a pattern match **themselves**: `cat` finds "cat". A small set of **metacharacters** have special meaning, and you put a backslash in front of one to match it literally: `\.` matches a real dot. The catch is that the backslash is usually processed **twice**: once by your language's string literal, then again by the regex engine. Raw strings (`r'…'`, `@'…'`, backticks) skip the first pass, which is why every example in this set uses them.

RULES OF THUMB

- Outside a class, the 12 metacharacters are `\ ^ $ . | ? * + ( ) [ {`
- `]`, `}` and `-` are literal outside a class
- Inside `[...]` only `] \ ^ -` are special (sheet 04)
- Escaping punctuation is safe in most flavors; JS `u` / `v` mode rejects needless ones like `\-` outside a class

ESCAPING IN PRACTICE (PYTHON)

```
import re
re.findall(r"3.14", "3.14 3x14")          # ['3.14', '3x14']
re.findall(r"3\.14", "3.14 3x14")         # ['3.14']
re.findall(r"\$\d+", "Cost: $25")         # ['$25']
re.findall(r"a\+b", "a+b aab")            # ['a+b']
re.escape("1+1=2?")                       # '1\\+1=2\\?'
re.findall(re.escape("(c)"), "(c) 2026")  # ['(c)']
len("\n"), len(r"\n")                     # (1, 2)
re.findall("\\\\", r"C:\temp")            # ['\\']
re.findall(r"\\", r"C:\temp")             # ['\\']
re.findall(r"\x41\u00e9", "Aé")           # ['Aé']
re.findall("\bcat\b", "a cat")            # []
re.findall(r"\bcat\b", "a cat")           # ['cat']
```

CODE EXPLAINED

- → `3.14` — unescaped `.` matches any character, so `3x14` matches too
- → `\$ \+` — escaped metacharacters are literal
- → `re.escape` — escapes every special character in a string
- → `"\\\\"` — non-raw: 4 backslashes in source = `\\` to the engine = one literal `\`
- → `\x41 \u00e9` — characters by hex code
- → `"\bcat\b"` — non-raw `\b` is a **backspace**, so nothing matches

METACHARACTERS

CHAR	MEANING
`.`	any char except newline
`^ $`	start / end anchors
`* + ?`	quantifiers: 0+, 1+, 0 or 1
`{ }`	counted quantifier `{2,5}`
`|`	alternation (or)
`( )`	group
`[ ]`	character class
`\`	escape / shorthand prefix

CHARACTER ESCAPES

ESCAPE	MATCHES
`\t \n \r`	tab, newline, carriage return
`\f \v`	form feed, vertical tab
`\xhh`	byte/char by hex: \x41 = A
`\uhhhh`	UTF-16 unit: JS, Java, .NET, Python
`\x{hhhh}`	code point: PCRE, Perl, Java
`\u{1F600}`	code point: JS with u, Ruby
`\0 \ooo`	NUL / octal (avoid; clashes with \1)
`\Q…\E`	quote a span: PCRE, Java, Perl

ESCAPE A LITERAL STRING

LANGUAGE	FUNCTION
Python	`re.escape(s)`
Java	`Pattern.quote(s)`
JavaScript	`RegExp.escape(s)` (ES2025)
C#	`Regex.Escape(s)`
Go	`regexp.QuoteMeta(s)`
Ruby	`Regexp.escape(s)`
PHP	`preg_quote($s, '/')`
Rust	`regex::escape(s)`

⚠ COMMON MISTAKES

- Not escaping the dot in domains and versions: `example.com` also matches `exampleXcom`
- Non-raw strings: `"\b"` is a backspace and `"\d"` may warn or break; use `r"\b"`
- Building a pattern from user input without escaping: `(` fails to compile and `.*` matches everything

✓ TIPS & BEST PRACTICES

- Escape any dynamic text with `re.escape` / `Pattern.quote` before inserting it
- `[.]` or `[$]` matches one literal metacharacter with no backslash, handy inside shell or JSON strings
- `\Q…\E` quotes a whole span in PCRE, Java and Perl

QUICK SUMMARY

- `\` + meta — match the metacharacter literally
- `\t \n \xhh \uhhhh` — character escapes
- raw strings — avoid double escaping
- `re.escape` & friends — quote untrusted text

How the Engine Matches

HOW A BACKTRACKING ENGINE WORKS

Most engines (PCRE, Java, JS, .NET, Python, Ruby) are **backtracking** engines. They try the pattern at text position 0; if it fails, they try position 1, and so on. At each position they follow the pattern left to right, and whenever a choice exists (a quantifier, an alternation) they take the first option and remember the rest. When a later piece fails, they **backtrack** to the last choice and try the next option. The first complete match wins, so results depend on the **order** of choices, not on which match is longest.

KEY CONSEQUENCES

- The **leftmost** match wins: an earlier start beats a longer match later
- Alternation is **ordered**: `cat|category` on "category" returns `cat`
- POSIX tools (`grep` , `awk`) pick the **leftmost-longest** match instead
- RE2, Go and Rust use automata: linear time, but no backreferences

TRACE: A.*C ON "XABCBCX"

```
# pos 0  'x' != a              -> fail, try next start
# pos 1  a ok; .* eats "bcbcx" (greedy, takes everything)
#        c? end of text        -> backtrack: .* gives back 'x'
#        c? 'x' != c           -> backtrack: .* gives back 'c'
#        c ok                  -> MATCH "abcbc" (span 1-6)
import re
re.search(r"a.*c", "xabcbcx").span()              # (1, 6)
re.search(r"cat|category", "category").group()    # 'cat'
re.search(r"category|cat", "category").group()    # 'category'
re.findall(r"aa", "aaaa")                         # ['aa', 'aa']
re.findall(r"(?=(aa))", "aaaa")                   # ['aa', 'aa', 'aa']
re.findall(r"a*", "baa")                          # ['', 'aa', '']
re.sub(r"x*", "-", "abc")                         # '-a-b-c-'
```

CODE EXPLAINED

- → Greedy `.*` grabs the rest of the line, then gives characters back one at a time
- → Ordered alternation: put the **longer** option first
- → Matches never overlap; the scan resumes after the previous match
- → `(?=(aa))` — a capturing lookahead finds overlapping matches
- → `a*` can match **zero** characters, so empty matches appear between letters

ENGINE TYPES

ENGINE	USED BY	BEHAVIOR
Backtracking NFA	PCRE, Perl, Java, JS, .NET, Python, Ruby	Tries paths one at a time; supports backrefs and lookaround; can blow up (sheet 15)
Automata (DFA / Pike VM)	RE2, Go, Rust, ripgrep, grep fast path	Tracks all paths at once; linear time; no backrefs or lookaround
POSIX NFA	C `regcomp`, some awk	Backtracks but must return the leftmost-longest match

THREE WAYS TO APPLY A PATTERN

MODE	EXAMPLES
search anywhere	`re.search`, Java `find()`, JS `test()`, `grep`
anchored at start	`re.match`, Java `lookingAt()`
whole string	`re.fullmatch`, Java `matches()`, `^…$`

⚠ COMMON MISTAKES

- Short alternative first: `in|int` matches only "in" inside "int"; order longest first or add `\b`
- Expecting `findall` to return overlapping matches; use a lookahead capture
- Using Python `re.match` when you meant search, or Java `matches()` when you meant find

✓ TIPS & BEST PRACTICES

- Use regex101's **debugger** to watch backtracking step by step
- Anchor with `^…$` or use a full-match API whenever you validate input
- Put the most common alternative first; it is tried first

QUICK SUMMARY

- `leftmost` — earliest start position wins
- `a|b` — tried in order, first success wins
- `backtracking` — retry the last choice on failure
- `RE2 / DFA` — linear time, fewer features

Character Classes & Shorthands

[abc] · [^abc] · [a-z0-9] · special chars in classes · \d \w \s · \D \W \S · . and newlines · [\s\S]

ONE CHARACTER FROM A SET

A **character class** `[...]` matches exactly **one** character from the set inside the brackets. Use ranges like a–z, put ^ first to negate the set, and combine freely: `[A-Za-z0-9_]`. **Shorthands** such as \d, \w and \s are ready-made classes, and their uppercase forms are the negations. Inside a class most metacharacters lose their meaning: `[.*+]` matches a literal dot, star or plus.

INSIDE THE BRACKETS

- Only `]`, `\`, `^` (first) and `-` (between chars) are special
- Literal `-`: put it first or last, `[-a]` / `[a-]`, or escape `\-`
- Literal `]`: escape it `[\]a]`; `[]a]` works in PCRE/Python but means an empty class in JS
- `[^...]` also matches **newlines**, unlike `.`

CLASSES & SHORTHANDS (PYTHON)

```python
import re
re.findall(r"[aeiou]", "regex")              # ['e', 'e']
re.findall(r"[^aeiou\s]", "a big cat")        # ['b', 'g', 'c', 't']
re.findall(r"gr[ae]y", "gray grey groy")      # ['gray', 'grey']
re.findall(r"[0-9A-Fa-f]+", "ff 1Z c0de")     # ['ff', '1', 'c0de']
re.findall(r"[\w.-]+@", "a.b-c@x.io")         # ['a.b-c@']
re.findall(r"[\]\\^-]", r"a]b\c^d-e")         # [']', '\\', '^', '-']
re.findall(r"\d+", "Order 66, gate 9")        # ['66', '9']
re.findall(r"\w+", "naïve café_2")            # ['naïve', 'café_2']
re.findall(r"\s", "a b\tc\n")                 # [' ', '\t', '\n']
re.findall(r".", "a\nb")                      # ['a', 'b']
re.findall(r"[\s\S]", "a\nb")                 # ['a', '\n', 'b']
```

CODE EXPLAINED

- → `[^aeiou\s]` — negated set: not a vowel and not whitespace
- → `[\w.-]` — shorthands work inside classes; `.` is literal there
- → `[\]\\^-]` — escaping the four special class characters
- → `\w` matched `ï` and `é`: Python 3 shorthands are Unicode-aware
- → `.` skips `\n`; `[\s\S]` matches anything, newlines included

CLASS SYNTAX

SYNTAX	MATCHES
`[abc]`	one of a, b, c
`[^abc]`	any char except a, b, c (incl. \n)
`[a-z]`	range by code point
`[a-zA-Z0-9_]`	several ranges combined
`[-.] [.-]`	literal hyphen at either edge
`[a^]`	^ is literal unless first
`[.*+?(){}|$]`	metacharacters are literal inside

SHORTHAND CLASSES

SHORTHAND	EQUIVALENT (ASCII)	NEGATION
`\d`	`[0-9]`	`\D`
`\w`	`[A-Za-z0-9_]`	`\W`
`\s`	`[ \t\n\r\f\v]`	`\S`
`.`	any except \n	`\n`
`\h \v`	horizontal / vertical space (PCRE, Java)	
`\R`	any line break \r\n \n \r (PCRE, Java)	
`\N`	any char but newline (PCRE, Perl)	

⚠ COMMON MISTAKES

- `[A-z]` looks like "all letters" but the range also includes `[ \ ] ^ _` and the backtick
- `[^…]` matches newlines, so `<[^>]*>` can run across lines (often what you want, sometimes not)
- Assuming `\d` means `[0-9]`: in Python 3 and .NET it also matches other scripts' digits, like ٣

✓ TIPS & BEST PRACTICES

- Prefer a negated class over `.*?` when you know the terminator: `"[^"]*"`
- Use `[0-9]` (or the ASCII flag) when you need ASCII-only digits
- `[\s\S]` or the `s` flag (sheet 12) lets `.` span lines

QUICK SUMMARY

- `[...]` — one character from the set
- `[^...]` — one character not in the set
- `\d \w \s` — digit, word char, whitespace
- `\D \W \S` — their negations

POSIX Classes & Unicode

BEYOND ASCII

`[a-z]` and `\w` were designed for English text. For everything else you have two tools. **POSIX bracket classes** like `[[:alpha:]]` are named sets used inside brackets by `grep`, `sed`, PCRE and Ruby. **Unicode properties** like `\p{L}` (any letter) or `\p{Script=Greek}` describe characters by their Unicode category or script, and work in nearly every modern flavor. Most need a Unicode switch: JS requires the `u` or `v` flag, and Python needs the third-party `regex` module.

UNICODE GOTCHAS

- `\w` and `\d` are ASCII in JS and Java (by default), Unicode in Python 3 and .NET
- One visible character can be several code points: `é` may be `e` + U+0301
- `\X` matches one **grapheme** (what a user sees as one character): PCRE, Perl, Java 9+, Ruby
- Normalize text first (NFC) so `é` has one form

UNICODE MATCHING (JAVASCRIPT)

```
"Ünïcödé 123".match(/\p{L}+/gu)              // ['Ünïcödé']
"Привет, world".match(/\p{Script=Cyrillic}+/gu)  // ['Привет']
"ABc".match(/\p{Lu}/gu)                      // ['A', 'B']
"1٢5".match(/\d/gu)                          // ['1', '5']
"1٢5".match(/\p{Nd}/gu)                      // ['1', '٢', '5']
"😀".match(/^.$/)                            // null
"😀".match(/^.$/u)                           // ['😀']
"é".normalize("NFD").length                  // 2
"aeixyz".match(/[\p{L}--[aeiou]]/gv)         // ['x', 'y', 'z']
```

CODE EXPLAINED

- → `\p{L}+` — any run of letters in any script
- → `\p{Script=Cyrillic}` — characters from one writing system
- → `\d` stays ASCII in JS; `\p{Nd}` matches every decimal digit
- → Without `u`, `.` sees an emoji as two UTF-16 halves
- → `v` flag — set subtraction `--` and intersection `&&`

POSIX BRACKET CLASSES

CLASS	MATCHES
`[:alpha:]`	letters
`[:digit:]`	digits 0–9
`[:alnum:]`	letters and digits
`[:upper:]` `[:lower:]`	upper / lower case
`[:space:]`	whitespace incl. newline
`[:blank:]`	space and tab only
`[:punct:]`	punctuation
`[:xdigit:]`	hex digits

UNICODE PROPERTIES

PROPERTY	MATCHES
`\p{L}`	any letter
`\p{Lu}` `\p{Ll}`	upper / lower case letter
`\p{N}` `\p{Nd}`	any number / decimal digit
`\p{P}` `\p{S}`	punctuation / symbol
`\p{Z}`	separator (spaces)
`\p{M}`	combining mark (accents)
`\p{Script=Han}`	a script (`\p{Han}` in PCRE, Go)
`\P{L}`	negation: not a letter

SUPPORT BY FLAVOR

FLAVOR	\P{…}	SET OPS
JavaScript	u / v flag	v: -- &&
Java	yes	`[a-z&&[^aeiou]]`
.NET	categories, `IsGreek` blocks	`[a-z-[aeiou]]`
Python re	no (regex module)	no
PCRE / Perl	yes	no
Ruby	yes	&&
Go / Rust	yes	Rust: && --

⚠ COMMON MISTAKES

- Using `[a-zA-Z]` for names: it rejects "José" and "Zoë"; use `\p{L}`
- Forgetting the `u` flag in JS: `\p{L}` is then a syntax error (or a literal `p` in old code)
- Comparing text with different normalization: NFC `é` and NFD `e` + ◌́ don't match each other

✓ TIPS & BEST PRACTICES

- Normalize input with NFC before matching user text
- In Python, `pip install regex` for `\p{…}`, `\X` and set operations
- Use `[[:alpha:]]` in `grep` / `sed`; it respects the locale, unlike `[a-z]`

QUICK SUMMARY

- `[[:alpha:]]` — POSIX named class, inside brackets
- `\p{L}` / `\P{L}` — Unicode property / negation
- `\p{Script=…}` — match a writing system
- `\X` — one user-visible character

Quantifiers

HOW MANY TIMES

A **quantifier** says how many times the item just before it may repeat. It applies to exactly **one atom**: a single character, an escape like `\d`, a class like `[a–z]`, or a whole group `( … )`. So `ab+` means "a, then one or more b", while `(ab)+` repeats the pair. By default quantifiers are **greedy**: they take as many repetitions as possible and give some back only if the rest of the pattern needs them (sheet 07).

GOOD TO KNOW

- `*` and `?` can match **zero** times, so they always succeed
- `{n,m}` has no spaces: `{2, 5}` is literal text in most flavors
- A `{` that isn't a valid quantifier is literal in Python, JS (non- `u`), PCRE; an error in Java and JS `u` mode
- A repeated capture group keeps only its **last** iteration

QUANTIFIERS IN ACTION (PYTHON)

```
import re
re.findall(r"colou?r", "color colour")       # ['color', 'colour']
re.findall(r"\d+", "a1b22c333")              # ['1', '22', '333']
re.findall(r"\d*", "a1")                     # ['', '1', '']
re.findall(r"\b\d{3}\b", "12 123 1234")      # ['123']
re.findall(r"\d{2,3}", "1 12 123 1234")      # ['12', '123', '123']
re.findall(r"\d{2,}", "1 12 12345")          # ['12', '12345']
re.findall(r"x{,2}", "xxx")                  # ['xx', 'x', '']
re.findall(r"(?:ab)+", "abababab abba")      # ['abababab', 'ab']
re.search(r"(ab)+", "abababab").group(1)     # 'ab'
re.search(r"(\d)+", "123").group(1)          # '3'
re.findall(r"x{a}", "x{a}")                  # ['x{a}']
```

CODE EXPLAINED

- → `u?` — the `u` is optional
- → `\d*` succeeds with zero digits, producing empty matches
- → `\d{2,3}` — 2 or 3 digits; "1234" yields "123" and leaves "4"
- → `{,2}` — 0 to 2 (Python, Ruby, newer PCRE2); literal or an error elsewhere
- → `(\d)+` — the group matched 1, 2, then 3; only `'3'` is kept
- → `x{a}` — not a valid quantifier, so the braces are literal

QUANTIFIERS

QUANTIFIER	MEANING	SAME AS
`*`	0 or more	`{0,}`
`+`	1 or more	`{1,}`
`?`	0 or 1 (optional)	`{0,1}`
`{n}`	exactly n	
`{n,}`	n or more	
`{n,m}`	n to m, inclusive	
`{,m}`	0 to m (not Java / JS)	`{0,m}`

WHAT THE QUANTIFIER BINDS TO

PATTERN	MATCHES
`ab+`	a, then 1+ b: ab, abbb
`(ab)+`	1+ copies of ab: ab, abab
`[ab]+`	any mix: a, ba, abba
`\d{3}-?\d{4}`	5551234 or 555–1234
`https?://`	`http://` or `https://`
`(\d{1,3}\.){3}\d{1,3}`	IPv4 shape: `10.0.0.1`
`\s*,\s*`	a comma with optional spaces

⚠ COMMON MISTAKES

- `ab*` when you meant `(ab)*`: the quantifier only covers `b`
- Expecting `(\w)+` to capture the whole word; wrap the quantifier inside: `(\w+)`
- `.*` at the start of a search pattern is redundant and slow; search already scans every position

✓ TIPS & BEST PRACTICES

- Use `(?:…)+` to repeat a sequence without creating a capture
- Bound open-ended counts on untrusted input: `\d{1,10}` rather than `\d+`
- Write `{0,m}` instead of `{,m}` for portable patterns

QUICK SUMMARY

- `* + ?` — 0+, 1+, 0 or 1
- `{n} {n,} {n,m}` — counted repetition
- `(…)+` — repeat a group
- repeated group — keeps the last capture only

Greedy, Lazy & Possessive

greedy · lazy *? +? ?? {n,m}? · ".*" vs ".*?" vs "[^"]*" · possessive *+ ++ ?+ · atomic (?>…)

THREE WAYS TO REPEAT

Greedy quantifiers (the default) take as much as they can, then give characters back one at a time if the rest of the pattern fails. Add ? to make them **lazy**: they take as little as possible and expand only when forced. Add + to make them **possessive**: they take as much as possible and **never** give anything back. An **atomic group** (?>…) applies the same no-give-back rule to a whole group. Lazy changes *which* match you get; possessive and atomic mostly change *speed* and prevent runaway backtracking.

CHOOSING

- Know the terminator? A negated class `[^"]*` beats both greedy and lazy
- Lazy is still a full match: `\d+?$` must reach the end, so it matches all digits
- Possessive can make a pattern **fail** that greedy would match
- No possessive or atomic in JS; Go and Rust don't need them (no backtracking)

GREEDY VS LAZY VS POSSESSIVE (PYTHON 3.11+)

```
import re
html = "<b>bold</b> and <i>it</i>"
re.findall(r"<.+>", html)            # ['<b>bold</b> and <i>it</i>']
re.findall(r"<.+?>", html)           # ['<b>', '</b>', '<i>', '</i>']
re.findall(r"<[^>]+>", html)         # ['<b>', '</b>', '<i>', '</i>']
s = '"a" and "b"'
re.findall(r'".*"', s)               # ['"a" and "b"']
re.findall(r'".*?"', s)              # ['"a"', '"b"']
re.findall(r'"[^"]*"', s)            # ['"a"', '"b"']
re.search(r"\d+?", "12345").group()   # '1'
re.search(r"\d+?$", "12345").group()  # '12345'
re.search(r'".*+"', s)               # None
re.search(r'"[^"]*+"', s).group()    # '"a"'
re.search(r"(?>a+)ab", "aaab")       # None
```

CODE EXPLAINED

- → `<.+>` — greedy runs to the **last** `>` on the line
- → `<.+?>` — lazy stops at the **first** `>`
- → `<[^>]+>` — can't cross `>`; fastest and clearest
- → `".*+"` — possessive `.*+` eats the closing quote and won't return it
- → `"[^"]*+"` — possessive is safe when the class can't eat the terminator
- → `(?>a+)` — atomic: keeps all three a's, so `ab` can't match

THE THREE FORMS

GREEDY	LAZY	POSSESSIVE
`*`	`*?`	`*+`
`+`	`+?`	`++`
`?`	`??`	`?+`
`{n,m}`	`{n,m}?`	`{n,m}+`
`{n,}`	`{n,}?`	`{n,}+`
`(…)`		`(?>…)`

BEHAVIOR

FORM	STRATEGY
`greedy`	Max first, then back off one at a time
`lazy`	Min first, then grow one at a time
`possessive`	Max, never back off
`atomic (?>…)`	Group matches once; no re-entry on backtrack

SUPPORT

FLAVOR	LAZY	POSS. / ATOMIC
`PCRE / Perl / Java`	yes	yes / yes
`Ruby`	yes	yes / yes
`Python re`	yes	3.11+ / 3.11+
`.NET`	yes	no / yes
`JavaScript`	yes	no / no
`Go / Rust`	yes	not needed

⚠ COMMON MISTAKES

- Reaching for `.*?` everywhere: it's slower than `[^x]*` and can still overshoot across lines
- Thinking lazy means "shortest overall"; it's shortest from the **leftmost** start position
- Making a quantifier possessive when it can also consume the next token (`.*+"`), so the match fails

✓ TIPS & BEST PRACTICES

- Default to negated classes, then lazy, then greedy, in that order of preference
- Use possessive or atomic on parts that can never need to give back, to stop catastrophic backtracking (sheet 15)
- In JS, emulate atomic with `(?=(…))\1`

QUICK SUMMARY

- `*` — greedy: max, then back off
- `*?` — lazy: min, then grow
- `*+` — possessive: max, never back off
- `(?>…)` — atomic group

Anchors & Word Boundaries

^ $ · multiline mode · \A \z \Z · $ before final \n · \b / \B · \< \> (GNU) · \G

MATCHING POSITIONS, NOT CHARACTERS

Anchors match a *position* between characters, so they consume nothing (they are **zero-width**). ^ and $ mark the start and end of the string, or of each line when **multiline** mode (`m`) is on. \A and \z always mean the absolute start and end, regardless of mode. **Word boundaries** \b match where a word character (\w) meets a non-word character or the edge of the text, which makes them the easiest way to match whole words.

GOTCHAS

- In most flavors `$` also matches **before a final `\n`**: `^\d+$` accepts "123\\n"
- JS, Go and Rust `$` (without `m`) means the true end only
- Ruby `^` and `$` are **always** line anchors; use `\A` and `\z` for validation
- `\b` uses `\w`, so in JS it's ASCII-only: `\bcafé\b` fails at `é`

ANCHORS & BOUNDARIES (PYTHON)

```python
import re
text = "first line\nsecond line\n"
re.findall(r"^\w+", text)               # ['first']
re.findall(r"^\w+", text, re.M)         # ['first', 'second']
re.findall(r"\w+$", text)               # ['line']
re.findall(r"line$", text, re.M)        # ['line', 'line']
bool(re.search(r"^\d+$", "123\n"))      # True
bool(re.search(r"\A\d+\Z", "123\n"))    # False
bool(re.fullmatch(r"\d+", "123\n"))     # False
re.findall(r"\bcat\b", "cat concat cat.")   # ['cat', 'cat']
re.findall(r"\Bcat\b", "cat concat")        # ['cat']
re.sub(r"\b", "|", "hi there")              # '|hi| |there|'
```

CODE EXPLAINED

- → Without `re.M`, `^` only matches at position 0
- → `\w+$` matched the last line: `$` allows the trailing `\n`
- → `^\d+$` accepts "123\\n"; `\A…\Z` (Python) or `fullmatch` rejects it
- → `\bcat\b` — whole word only; skips "concat"
- → `\Bcat` — "cat" **inside** a word
- → `\b` is zero-width: inserting at it marks each word edge

ANCHORS

ANCHOR	MATCHES AT
`^`	start of string; start of each line with `m`
`$`	end (or before final \n); end of each line with `m`
`\A`	start of string, always (not JS)
`\z`	very end of string (PCRE, Java, .NET, Ruby, Go)
`\Z`	end or before final \n; **Python: very end**
`\b`	word boundary (\w next to \W or edge)
`\B`	not a word boundary
`\G`	where the previous match ended (PCRE, Java, .NET)
`\< \>`	start / end of word (GNU grep, sed, vim)

$ WITHOUT MULTILINE

FLAVOR	$ MATCHES
PCRE, Perl, PHP	end or before final \n
Java, .NET	end or before final line break
Python	end or before final \n
JavaScript	very end only
Go, Rust	very end only
Ruby	end of **any** line
POSIX grep / sed	end of each line (line-based)

⚠ COMMON MISTAKES

- Validating with `^…$` in Ruby: "ok\\n<script>" passes because `$` matches at the first line end
- Forgetting `m` and wondering why `^` matches only once in a multi-line string
- Using `\b` around non-word characters: `\b\$5\b` fails because `$` isn't a word character

✓ TIPS & BEST PRACTICES

- Validate whole input with `\A…\z` or a full-match API (`fullmatch`, `matches()`)
- Use `\b` for whole-word search and replace: `\bid\b` won't touch "valid"
- Anchoring a pattern lets the engine fail fast instead of trying every position

QUICK SUMMARY

- `^ $` — string or line edges (with `m`)
- `\A \z` — absolute start and end
- `\b \B` — word boundary / not a boundary
- anchors are zero-width — they consume nothing

Groups & Alternation

(…) capture · | lowest precedence · ^a|b$ pitfall · (?:…) · numbering by (· nested groups · (cat|dog)s?

GROUPING AND CHOOSING

Parentheses do two jobs: they **group** part of a pattern so a quantifier or alternation applies to all of it, and they **capture** the text that part matched so you can read it back or reuse it. `|` means "or" and has the **lowest precedence** of any operator, so it splits the entire pattern (or the enclosing group) into alternatives. Use a non-capturing group `(?:…)` when you need grouping but not the captured text.

RULES

- Groups are numbered by their **opening** parenthesis, left to right, starting at 1
- Group 0 is the whole match
- A group that didn't take part in the match returns `None` / `null` / empty
- `[abc]` beats `a|b|c` for single characters: faster and clearer

GROUPS & ALTERNATION (PYTHON)

```
import re
m = re.search(r"(\d{4})-(\d{2})-(\d{2})", "Due 2026-10-10")
m.group(0)                                  # '2026-10-10'
m.group(1), m.group(3)                      # ('2026', '10')
m.groups()                                  # ('2026', '10', '10')
re.findall(r"^cat|dog$", "cat or dog")      # ['cat', 'dog']
re.findall(r"^(?:cat|dog)$", "cat or dog")  # []
re.findall(r"(cat|dog)s?", "cats dog")      # ['cat', 'dog']
re.findall(r"(?:cat|dog)s?", "cats dog")    # ['cats', 'dog']
re.search(r"((a)(b))", "ab").groups()       # ('ab', 'a', 'b')
re.search(r"(x)?y", "y").group(1)           # None
```

CODE EXPLAINED

- → `(\d{4})` — captures the year as group 1
- → `^cat|dog$` is `(^cat)|(dog$)`, not "cat or dog as the whole string"
- → `^(?:cat|dog)$` — the group keeps the anchors around both choices
- → `findall` returns **group** text when the pattern has groups; `(?:…)` avoids that
- → Nested `((a)(b))` — outer group is 1, then 2 and 3
- → `(x)?` didn't participate, so group 1 is `None`

GROUP SYNTAX

SYNTAX	MEANING
`(…)`	capture group, numbered
`(?:…)`	group without capturing
`a|b`	a or b (whole pattern / group)
`(…)?`	optional group
`(…){2}`	group repeated twice
`(?<name>…)`	named group (sheet 10)
`(?>…)`	atomic group (sheet 07)

NUMBERING: ((A)(B(C)))

GROUP	OPENED BY	TEXT
0	whole match	abc
1	1st (	abc
2	2nd (	a
3	3rd (	bc
4	4th (	c

PRECEDENCE

PATTERN	PARSED AS
`^a|b$`	`(^a)|(b$)`
`ab|cd`	`(ab)|(cd)`
`a(b|c)d`	abd or acd
`x|y+`	x or (y+)
`gr(a|e)y`	gray or grey

⚠ COMMON MISTAKES

- Unwrapped alternation with anchors: `^yes|no$` accepts "yes please" and "say no"; wrap it: `^(?:yes|no)$`
- Adding a group for grouping only and shifting every later group number; use `(?:…)`
- Expecting `findall` to return full matches when the pattern has a capture group

✓ TIPS & BEST PRACTICES

- Default to `(?:…)` and capture only what you'll read back
- Use named groups once a pattern has more than two or three captures
- Factor shared prefixes: `\b(?:Jan|Feb|Mar)\b` rather than repeating `\b` in each branch

QUICK SUMMARY

- `(…)` — group + capture
- `(?:…)` — group only
- `|` — lowest precedence: splits the whole group
- `group(0)` — the entire match

Named Groups & Backreferences

(?<name>…) · (?P<name>…) · \1 \2 · \k<name> / (?P=name) · \g{-1} · branch reset (?|…) · doubled words

NAMING AND REUSING CAPTURES

A **named group** labels a capture so you can read it by name instead of counting parentheses: `(?<year>\d{4})`. A **backreference** matches the *same text* a group already captured, later in the same pattern: `(\w+) \1` matches a doubled word. Note the difference: a backreference repeats the captured **text**, not the subpattern; `(\d)\1` matches "77" but not "78". Named groups are still numbered, so `\1` and `\k<name>` both work.

FLAVOR NOTES

- Python `re` uses `(?P<name>…)` and `(?P=name)` only
- JS, Java, .NET, PCRE, Ruby and Go use `(?<name>…)`
- Go and Rust (RE2-style) have **no backreferences** at all
- Duplicate group names: .NET, PCRE `(?J)`, JS ES2025 (in different branches)

NAMED GROUPS & BACKREFERENCES (PYTHON)

```
import re, regex
m = re.search(r"(?P<year>\d{4})-(?P<mon>\d\d)", "on 2026-10")
m.group("year")                                  # '2026'
m.groupdict()                                    # {'year': '2026', 'mon': '10'}
re.findall(r"\b(\w+) \1\b", "the the cat sat sat")  # ['the', 'sat']
re.sub(r"\b(\w+) \1\b", r"\1", "the the cat")       # 'the cat'
re.search(r"(['\"]).*?\1", "say 'hi' now").group()  # "'hi'"
re.search(r"<(?P<t>\w+)>.*?</(?P=t)>", "<b>x</b>")[0]   # '<b>x</b>'
[bool(re.fullmatch(r"(\d)\1", t)) for t in ("77", "78")]  # [True, False]
regex.search(r"(?<y>\d+)", "42").group("y")       # '42'
regex.search(r"(?|(\d+)|([a-z]+))", "abc").group(1)   # 'abc'
```

CODE EXPLAINED

- → `(?P<year>…)` + `.group("year")` — read by name
- → `groupdict()` — every named group as a dict
- → `\b(\w+) \1\b` — a word followed by the **same** word
- → `(['"]).*?\1` — the closing quote must match the opening one
- → `(?P=t)` — Python's named backreference
- → `(?|…)` — branch reset: each branch reuses group 1 (PCRE, Perl, `regex`)

DEFINE A NAMED GROUP

SYNTAX	FLAVORS
`(?<name>…)`	PCRE, Java, .NET, JS, Ruby, Go, `regex`
`(?P<name>…)`	Python, PCRE, Go, Rust
`(?'name'…)`	.NET, PCRE, Ruby, Perl

BACKREFERENCE IN THE PATTERN

SYNTAX	FLAVORS
`\1 … \9`	all backtracking flavors
`\k<name>`	PCRE, Java, .NET, JS, Ruby, Perl
`(?P=name)`	Python, PCRE
`\g{-1} \g1`	relative / absolute (PCRE, Perl)

READ A NAMED CAPTURE

LANGUAGE	CODE
Python	`m.group("name")`, `m["name"]`
JavaScript	`m.groups.name`
Java	`m.group("name")`
C#	`m.Groups["name"].Value`
Ruby	`m[:name]`
Go	`m[re.SubexpIndex("name")]`

⚠ COMMON MISTAKES

- Expecting `(\d)\1` to mean "two digits": it means the **same** digit twice; use `\d{2}`
- Using `(?<name>…)` in Python `re`: it's a syntax error; write `(?P<name>…)`
- Writing `\10` and getting group 1 + "0" (or octal); use `\g<10>` / `\g{10}` for group 10+

✓ TIPS & BEST PRACTICES

- Name every group you read back in code; numbers break when the pattern changes
- Pair backreferences with lazy or negated content: `(['"])[^'"]*\1`
- In replacements, named groups are `\g<name>` (Python), `${name}` (Java, .NET), `$<name>` (JS)

QUICK SUMMARY

- `(?<name>…)` — named capture
- `\1` / `\k<name>` — repeat the captured text
- `(?P<name>…)` / `(?P=name)` — Python forms
- `(?|…)` — branch reset shares numbers

Lookahead & Lookbehind

(?=…) · (?!…) · (?<=…) · (?<!…) · zero-width · lookbehind length limits · \K · stacked lookaheads

LOOK WITHOUT CONSUMING

A **lookaround** checks whether a pattern matches next to the current position, then returns to that position without consuming anything. **Lookahead** `(?=…)` peeks to the right, **lookbehind** `(?<=…)` peeks to the left, and the `!` forms assert that the pattern does **not** match. Because they're zero-width, lookarounds let you match text based on its surroundings while keeping those surroundings out of the result, and several lookaheads at one position act like an **AND** of conditions.

LIMITS

- Lookbehind length is restricted in many flavors (table below)
- Groups inside a lookaround still capture: `(?=(\w+))`
- Go and Rust have **no** lookaround; restructure or post-filter in code
- `\K` (PCRE, Perl, Ruby, `regex`) drops everything matched so far: a lookbehind without length limits

LOOKAROUNDS (PYTHON)

```python
import re, regex
re.findall(r"\d+(?=px)", "10px 20em 30px")        # ['10', '30']
re.findall(r"\d+(?!px|\d)", "10px 20em 30px")     # ['20']
re.findall(r"(?<=\$)\d+", "$5 and €7 and $12")    # ['5', '12']
re.findall(r"(?<!\$)\b\d+", "$5 and 7 and $12")   # ['7']
re.sub(r"\B(?=(\d{3})+(?!\d))", ",", "1234567")   # '1,234,567'
strong = r"(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}"
bool(re.fullmatch(strong, "Passw0rd"))            # True
bool(re.fullmatch(strong, "password"))            # False
re.search(r"(?<=\w+)@", "me@x")                   # re.error: fixed width only
regex.search(r"(?<=\w+)@", "me@x").start()        # 2
regex.sub(r"foo\Kbar", "BAZ", "foobar")           # 'fooBAZ'
```

CODE EXPLAINED

- → `(?=px)` — digits only if `px` follows; `px` isn't returned
- → `(?!px|\d)` — the `\d` stops `\d+` backing off to `"1"` in `10px`
- → `(?<=\$)` — digits preceded by a dollar sign
- → `\B(?=(\d{3})+(?!\d))` — positions followed by groups of 3 digits
- → Stacked lookaheads at the start: digit AND lower AND upper
- → `\K` — keep `foo` out of the match, so only `bar` is replaced

SYNTAX

SYNTAX	SUCCEEDS IF
`(?=X)`	X matches next (lookahead)
`(?!X)`	X does not match next
`(?<=X)`	X matches just before (lookbehind)
`(?<!X)`	X does not match just before
`\K`	reset match start here

LOOKBEHIND LENGTH

FLAVOR	ALLOWED
`JavaScript, .NET`	any length
`Python regex module`	any length
`Java`	bounded: `{1,10}` ok, `+` `*` not
`PCRE2, Perl, Ruby`	fixed-length alternatives `(?<=a|bc)`
`Python re`	fixed width only
`Go, Rust`	none

COMMON USES

PATTERN	FINDS
`\d+(?=%)`	number before a %
`(?<=@)[\w.]+`	domain after @
`^(?!.*admin).*$`	lines without “admin”
`q(?!u)`	q not followed by u
`(?<![\w.])\d+(?![\w.])`	standalone integers

⚠ COMMON MISTAKES

- Expecting lookaround text in the match: `(?<=\$)\d+` returns `5`, not `$5`
- A negative lookahead without an anchor: `(?!foo)\w+` still matches “oo” inside “foo”; use `\b(?!foo\b)\w+`
- Porting a lookbehind-heavy pattern to Go or Rust; it won't compile there

✓ TIPS & BEST PRACTICES

- Use `^(?=…)(?=…)` to require several independent conditions in any order
- Replace a variable-length lookbehind with a capture group plus `\1` in the replacement
- Prefer `\K` over lookbehind in PCRE for speed and no length limits

QUICK SUMMARY

- `(?=X)` / `(?!X)` — X does / doesn't follow
- `(?<=X)` / `(?<!X)` — X does / doesn't precede
- zero-width — nothing is consumed
- `\K` — discard the match so far

Flags & Inline Modifiers

CHANGING HOW A PATTERN BEHAVES

Flags (also called modifiers or options) change how the whole pattern is interpreted: case sensitivity, what ^ $ and `.` mean, whether whitespace in the pattern is ignored, and how Unicode is handled. You set them outside the pattern (`/x/gi` in JS, `re.I` in Python, `Pattern.CASE_INSENSITIVE` in Java) or **inline** at the start of the pattern with `(?i)`. Inline groups like `(?i:…)` limit a flag to part of the pattern.

NAMING TRAPS

- `m` changes `^` and `$` only; it does **not** let `.` match newlines (that's `s`)
- .NET calls the `s` flag **Singleline**: it's the same dot-all behavior
- Ruby's `/m` means dot-all; Ruby `^` `$` are always per line
- Python 3.11+: global `(?i)` must be at the very start of the pattern

FLAGS IN PYTHON

```python
import re
re.findall(r"hello", "Hello HELLO", re.I)      # ['Hello', 'HELLO']
re.findall(r"(?i)hello", "Hello HELLO")        # ['Hello', 'HELLO']
re.findall(r"^\w", "a\nb", re.M)               # ['a', 'b']
re.findall(r"a.b", "a\nb", re.S)               # ['a\nb']
re.findall(r"\d", "1१", re.A)                  # ['1']
re.findall(r"(?i:A)b", "ab Ab AB")             # ['ab', 'Ab']
re.findall(r"x", "xX", re.I | re.M)            # ['x', 'X']
pat = re.compile(r"""
    (?P<area>\d{3})   # area code
    [-.\s]?           # optional separator
    (?P<num>\d{4})    # number
""", re.X)
pat.search("call 555 1234")["num"]             # '1234'
```

INLINE MODIFIERS

SYNTAX	EFFECT
`(?i)`	on for the whole pattern (not JS; Python: start only)
`(?i:…)`	on inside this group only (JS: ES2025)
`(?-i)`	off from here on (PCRE, Java, .NET, Ruby)
`(?-i:…)`	off inside this group (Python 3.7+ too)
`(?im-sx)`	several on and off at once
`(?x)`	verbose mode from inside the pattern

FLAGS ACROSS LANGUAGES

FLAG	EFFECT	PYTHON	JAVASCRIPT	JAVA	.NET REGEXOPTIONS
`i`	case-insensitive	`re.I`	`i`	`CASE_INSENSITIVE`	`IgnoreCase`
`m`	^ $ per line	`re.M`	`m`	`MULTILINE`	`Multiline`
`s`	`.` matches \n	`re.S`	`s`	`DOTALL`	`Singleline`
`x`	verbose, # comments	`re.X`	none	`COMMENTS`	`IgnorePatternWhitespace`
`u`	Unicode mode	`default (re.A = ASCII)`	`u / v`	`UNICODE_CHARACTER_CLASS`	`default`
`g`	find all matches	`findall / finditer`	`g`	`while (m.find())`	`Matches()`
`y`	sticky: match at `lastIndex` only	`pattern.match(s, pos)`	`y`	`\G or region()`	`\G`
`n`	only named groups capture	none	none	none	`ExplicitCapture`

⚠ COMMON MISTAKES

- Adding `m` to make `.` span lines; you needed `s` (or `[\s\S]`)
- Reusing a JS regex with `g` across `test()` calls; `lastIndex` makes alternate calls fail (sheet 20)
- In verbose mode, forgetting that a literal space or `#` must be escaped

✓ TIPS & BEST PRACTICES

- Use verbose mode for any pattern longer than ~40 characters, with a comment per piece
- Prefer scoped `(?i:…)` over a global flag when only one word is case-insensitive
- Combine flags with `|` in Python and Java: `re.I | re.M`

QUICK SUMMARY

- `i m s x` — case, lines, dot-all, verbose
- `g y` — JS: all matches / sticky
- `(?i)` — inline flag
- `(?i:…)` — flag for one group only

Substitution & Replacement Syntax

$1 vs \1 vs \g<1> · ${name} / \g<name> / $<name> · $& $0 \0 & · $` $' · $$ · \U \L \E · ${1}0

THE REPLACEMENT STRING IS ITS OWN MINI-LANGUAGE

In search-and-replace, the **replacement** is not a regex, but it has its own escape syntax for inserting captured text, and that syntax varies more between languages than the patterns do. Python uses backslashes (`\1`, `\g<name>`), most others use dollars (`$1`, `${name}`), and `sed` uses `\1` and `&`. When the text you insert comes from users, escape `$` and `\` in it, or pass a **function** that returns the replacement, which skips the mini-language entirely.

GOOD TO KNOW

- Python and sed: `$` is literal. JS, Java, .NET: `$` is special
- Java treats `\` as an escape in replacements: `\\` for one backslash
- Case conversion `\U…\E` exists in Perl, GNU sed, PCRE2 extended mode, VS Code and Notepad++
- Function replacements are the escape hatch for anything conditional

REPLACING (PYTHON)

```python
import re
d = "2026-03-14"
re.sub(r"(\d+)-(\d+)-(\d+)", r"\3/\2/\1", d)          # '14/03/2026'
re.sub(r"(?P<y>\d+)-(?P<m>\d+)-(?P<d>\d+)", r"\g<d>.\g<m>.\g<y>", d)  # '14.03.2026'
re.sub(r"(\d)", r"\g<1>0", "a1b2")                    # 'a10b20'
re.sub(r"\w+", r"[\g<0>]", "hi there")                # '[hi] [there]'
re.sub(r"\w+", lambda m: m[0].upper(), "hi there")    # 'HI THERE'
re.sub(r"(\w)(\w*)", lambda m: m[1].upper() + m[2], "hi there")  # 'Hi There'
re.sub(r"x", "$1", "x")                               # '$1'
# JavaScript  d.replace(/(\d+)-(\d+)-(\d+)/, "$3/$2/$1")
# Java        d.replaceAll("(\\d+)-(\\d+)-(\\d+)", "$3/$2/$1")
# C#          Regex.Replace(d, @"(\d+)-(\d+)-(\d+)", "$3/$2/$1")
# GNU sed     sed -E 's/([0-9]+)-([0-9]+)-([0-9]+)/\3\/\2\/\1/'
# Perl        s/(\w+)/\u$1/g   # capitalize each word
```

CODE EXPLAINED

- → `\3/\2/\1` — reorder captured groups
- → `\g<name>` — Python's named reference in a replacement
- → `\g<1>0` — group 1 then a literal `0` (plain `\10` would mean group 10)
- → `\g<0>` — the whole match (`$&` / `$0` elsewhere)
- → `lambda m: …` — compute each replacement in code
- → `$1` is literal in Python; it only means group 1 in `$` flavors

REPLACEMENT TOKENS BY LANGUAGE

INSERT	PYTHON	JAVASCRIPT	JAVA	.NET	PERL / GNU SED
group 1	`\1 \g<1>`	`$1`	`$1`	`$1`	`$1 / \1`
named group	`\g<name>`	`$<name>`	`${name}`	`${name}`	`$+{name} / none`
whole match	`\g<0>`	`$&`	`$0`	`$0 $&`	`$& / &`
before / after match	none	`$` `$'`	none	`$` `$'`	`$` `$'` / none
group 1 then digit 0	`\g<1>0`	`$010`	`$10 (if < 10 groups)`	`${1}0`	`${1}0 / \10`
literal $	`$`	`$$`	`\$`	`$$`	`\$ / $`
literal \	`\\`	`\`	`\\`	`\`	`\\`
upper / lower case	function	function	function	function	`\U \L \u \l … \E`

⚠ COMMON MISTAKES

- Inserting user text into a `$`-style replacement: a `$1` in their input becomes a group reference
- Writing `r"\10"` in Python for group 1 + "0"; it's group 10 (or an error)
- Using Python's `\1` syntax in JS, or JS's `$1` in Python; both are treated literally

✓ TIPS & BEST PRACTICES

- Escape literal replacement text: `Matcher.quoteReplacement(t)` (Java), `t.Replace("$", "$$")` (C#), or pass `() => t` (JS)
- Use a replacement **function** whenever the output depends on the match
- Prefer named groups in long replacements so a pattern edit doesn't scramble the numbers

QUICK SUMMARY

- `\1` / `$1` — group 1 (Python, sed / most others)
- `\g<name>` / `${name}` / `$<name>` — named group
- `\g<0>` / `$&` / `$0` / `&` — whole match
- function — compute the replacement

Advanced Constructs

(?(1)yes|no) · (?R) / (?1) · balanced parentheses · (?&name) · (*SKIP) (*FAIL) · (*UCP) · .NET (?<-open>) · (?#…)

BEYOND REGULAR LANGUAGES

PCRE, Perl and a few other flavors add constructs that go past what "regular" expressions can describe. **Conditionals** choose a branch based on whether a group matched. **Recursion** and **subroutine calls** reuse a pattern (not its captured text), which is enough to match nested structures like balanced parentheses. **Backtracking verbs** such as `(*SKIP)(*FAIL)` control the engine directly. These are powerful but non-portable; check the support table before relying on one.

WHEN TO USE THEM

- Nested brackets, quotes inside code, or "match X except inside Y" problems
- Python: most of these need the third-party `regex` module (examples below)
- No support in JS, Java, Go or Rust; use a parser or code there
- If a pattern needs more than one of these, a real parser is usually clearer

ADVANCED CONSTRUCTS (PYTHON REGEX MODULE, PCRE SYNTAX)

```
import regex
# Balanced parentheses: recurse into the whole pattern
regex.findall(r"\((?:[^()]++|(?R))*\)", "f(a(b)c) g(d)")   # ['(a(b)c)', '(d)']
# Conditional: if group 1 matched "<", require ">"
ok = lambda p, s: bool(regex.fullmatch(p, s))
pat = r"(<)?\w+(?(1)>)"
[ok(pat, s) for s in ("<a>", "a", "<a")]                    # [True, True, False]
# Subroutine: reuse a named group's pattern, not its text
ip = r"(?<o>25[0-5]|2[0-4]\d|1?\d?\d)(?:\.(?&o)){3}"
ok(ip, "10.0.0.255"), ok(ip, "10.0.0.256")                  # (True, False)
# Skip quoted strings, match words everywhere else
regex.findall(r'"[^"]*"(*SKIP)(*FAIL)|\w+', 'a "b c" d')   # ['a', 'd']
# Comments inside a pattern
regex.findall(r"\d+(?# digits)%", "50% of 8")              # ['50%']
```

CODE EXPLAINED

- → `(?R)` — recurse into the entire pattern at this point
- → `[^()]++` — possessive: never re-splits text between parens
- → `(?(1)>)` — require `>` only if group 1 matched
- → `(?&o)` — run group `o`'s pattern again (each octet re-validated)
- → `(*SKIP)(*FAIL)` — match the unwanted part, then discard it and resume after it
- → `(?#…)` — an inline comment

CONSTRUCTS & SUPPORT

SYNTAX	MEANING	SUPPORTED BY
`(?(1)yes|no)`	conditional on group 1 (or `(?(<name>)…)`)	PCRE, Perl, .NET, Python `re`, Ruby
`(?R)` `(?0)`	recurse the whole pattern	PCRE, Perl, `regex`; Ruby `\g<0>`
`(?1)` `(?&name)`	subroutine: call group's pattern	PCRE, Perl, `regex`; Ruby `\g<name>`
`(?(DEFINE)…)`	define named subpatterns, match nothing	PCRE, Perl, `regex`
`(*SKIP)(*FAIL)`	discard what matched, skip past it	PCRE, Perl, `regex`
`(*PRUNE)` `(*COMMIT)` `(*ACCEPT)`	other backtracking-control verbs	PCRE, Perl
`(*UCP)` `(*UTF)` `(*CRLF)`	start-of-pattern engine options	PCRE2
`(?<open>\(\) … (?<-open>\))`	balancing groups: push / pop a stack	.NET only
`(?#comment)`	inline comment	PCRE, Perl, Python, .NET, Ruby

⚠ COMMON MISTAKES

- Using `(?R)` with unbounded alternatives: without possessive or atomic parts, nested input backtracks exponentially
- Expecting a subroutine `(?1)` to match the same **text** like `\1`; it re-runs the **pattern**
- Porting a `(*SKIP)(*FAIL)` pattern to JS or Java; it fails to compile

✓ TIPS & BEST PRACTICES

- Wrap recursive pieces in `[^…]++` or `(?>…)` to keep them fast
- Use `(?(DEFINE)…)` to name building blocks, then compose with `(?&name)`
- In .NET, balanced matching uses `(?<open>…)`, `(?<-open>…)` and a final `(?(open)(?!))`

QUICK SUMMARY

- `(?(1)a|b)` — conditional branch
- `(?R)` / `(?1)` — recursion and subroutines
- `(*SKIP)(*FAIL)` — match everything except X
- non-portable — PCRE, Perl, `regex`, .NET

Performance & Catastrophic Backtracking

ReDoS · (a+)+ · (a|a)* · atomic / possessive fixes · anchor & fail fast · precompile · timeouts · RE2 / NonBacktracking

WHEN A REGEX NEVER FINISHES

A backtracking engine that fails a match must try **every** way the pattern could have split the text before giving up. When a pattern has **nested or overlapping quantifiers**, like `(a+)+` or `(\w|\d)*`, the number of ways grows exponentially with input length. A 30-character string can then take minutes. This is **catastrophic backtracking**, and when attackers can supply the input it's a denial-of-service bug called **ReDoS**. The fix is to make each character matchable in only one way.

REAL INCIDENTS

- Stack Overflow, 2016: a whitespace-trim regex hit a post with ~20,000 spaces and took the site down
- Cloudflare, 2019: `.*(?:.*=.*)` in a WAF rule pinned CPUs worldwide for 27 minutes
- Go, Rust and RE2 are immune: they run in linear time

CATASTROPHIC BACKTRACKING (PYTHON)

```
import re, time
evil = re.compile(r"^(a+)+$")
for n in (20, 22, 24):
    t = time.perf_counter(); evil.match("a" * n + "!")
    print(n, round(time.perf_counter() - t, 2))
# 20 0.08   22 0.19   24 0.85   <- ~2x per extra character
re.match(r"^a+$", "a" * 10_000 + "!")      # None
re.match(r"^(?>a+)+$", "a" * 10_000 + "!") # None
re.match(r"^(a++)+$", "a" * 10_000 + "!")  # None
# Python regex module: timeout as a safety net
import regex
regex.match(r"^(a|a)*$", "a" * 30 + "!", timeout=0.5)  # TimeoutError
```

CODE EXPLAINED

- → `(a+)+` — "aaaa" can split as 4, 3+1, 2+2, 1+1+1+1…: 2^{n-1} ways
- → The trailing `!` forces failure, so every split is tried
- → `^a+$` — same strings, one way to match; instant
- → `(?>a+)` / `a++` — atomic / possessive: no re-splitting
- → `timeout=` — the `regex` module raises instead of hanging

DANGEROUS SHAPE → FIX

DANGEROUS	WHY	SAFER
`(a+)+`	nested quantifier	`a+`
`(\w|\d)+`	overlapping alternatives	`\w+`
`(.*,)*`	`.*` can also eat commas	`(?:[^,]*,)*`
`^(\w+\s?)*$`	optional separator	`^\w+(?:\s\w+)*$`
`(a|a)*`	identical alternatives	`a*`

DEFENSES BY LANGUAGE

LANGUAGE	OPTION
.NET	`matchTimeout` arg; `RegexOptions.NonBacktracking` (.NET 7+)
Python	`regex` module `timeout=`; bound input length
Ruby 3.2+	`Regexp.timeout = 1.0`; built-in memoization
Java	no timeout: run in a worker with `Future.get(timeout)`
JavaScript	no timeout: use the `re2` package or a worker
PHP	`pcre.backtrack_limit`; `preg_*` then fails (`false` / `null`)
Go, Rust	linear time by design

⚠ COMMON MISTAKES

- Testing only with matching input; ReDoS appears when the match **fails** near the end
- Recompiling inside a loop: Java `String.matches()` compiles the pattern on every call
- Starting a search pattern with `.*`: the engine already tries every position, so it's pure extra work

✓ TIPS & BEST PRACTICES

- Make each character belong to exactly one quantifier; use tight classes like `[^,]*`
- Compile once (`re.compile` , `Pattern.compile` , a `static readonly Regex`) and reuse
- Cap input length before matching untrusted text, and lint patterns with a ReDoS checker

QUICK SUMMARY

- `(a+)+` — nested quantifiers = exponential
- `(?>…)` / `a++` — stop re-splitting
- timeout / linear engine — safety net
- precompile — pay the parse cost once

Python re — Core Functions

r"\d+" · search vs match vs fullmatch · findall · finditer · split · re.compile + flags · Match .group() / .span()

THE RE MODULE

Python's built-in `re` module offers module-level functions (`re.search(pattern, text)`) and compiled **Pattern** objects (`re.compile(pattern).search(text)`) with the same methods. A successful match returns a **Match** object, which is truthy and holds the matched text, groups and positions; a failed one returns `None`. Always write patterns as **raw strings** so backslashes reach the engine untouched.

KEY POINTS

- `search` scans anywhere, `match` only at position 0, `fullmatch` needs the whole string
- `findall` returns **strings**, or **tuples of groups** if the pattern has groups
- Module functions cache recently used patterns; compile when you reuse one in a loop
- Check for `None` before `.group()`: `if m := re.search(…):`

CORE API

```
import re
s = "Ann 31, Bob 27, Cy 45"
re.search(r"\d+", s)                    # <re.Match object; span=(4, 6), match='31'>
re.match(r"\d+", s)                     # None
re.findall(r"\d+", s)                   # ['31', '27', '45']
re.findall(r"(\w+) (\d+)", s)           # [('Ann', '31'), ('Bob', '27'), ('Cy', '45')]
[m.start() for m in re.finditer(r"\d+", s)]   # [4, 12, 19]
re.split(r",\s*", s)                    # ['Ann 31', 'Bob 27', 'Cy 45']
age = re.compile(r"(?P<name>\w+) (?P<age>\d+)", re.I)
m = age.search(s)
m.group("name"), m["age"]               # ('Ann', '31')
m.groups(), m.span(), m.end(2)          # (('Ann', '31'), (0, 6), 6)
```

CODE EXPLAINED

- → `match` — anchored at 0; `"Ann…"` doesn't start with a digit
- → `findall` with 2 groups → list of 2-tuples
- → `finditer` — Match objects, for positions and groups
- → `split` — break on the pattern; captured delimiters are kept
- → `(?P<name>…)` + `m["age"]` — groups by name

MODULE FUNCTIONS

FUNCTION	RETURNS
`search(p, s)`	first Match anywhere, or None
`match(p, s)`	Match at position 0, or None
`fullmatch(p, s)`	Match of the whole string, or None
`findall(p, s)`	list of `str` (or tuples of groups)
`finditer(p, s)`	iterator of Match objects
`split(p, s, maxsplit=0)`	list of pieces
`compile(p, flags)`	reusable Pattern object

MATCH OBJECT

ATTRIBUTE	GIVES
`m.group() / m[0]`	whole match
`m.group(1, 2)`	tuple of groups
`m.groups(default)`	all groups
`m.groupdict()`	named groups as dict
`m.start(g) m.end(g)`	index of group g
`m.span(g)`	(start, end)

FLAGS

FLAG	INLINE
`re.I IGNORECASE`	(?i)
`re.M MULTILINE`	(?m)
`re.S DOTALL`	(?s)
`re.X VERBOSE`	(?x)
`re.A ASCII`	(?a)
`re.DEBUG`	prints parse tree

⚠ COMMON MISTAKES

- Calling `.group()` on a failed search: `AttributeError: 'NoneType' object has no attribute 'group'`
- Adding a group to a `findall` pattern and suddenly getting tuples or partial text; use `(?:…)`
- Using `re.match` to search mid-string; it only checks position 0

✓ TIPS & BEST PRACTICES

- Use the walrus: `if m := pat.search(line): …`
- `finditer` over `findall` when you need positions or large inputs (lazy)
- `re.DEBUG` (or regex101's Python flavor) shows exactly how Python parsed a pattern

QUICK SUMMARY

- `search` / `match` / `fullmatch` — anywhere / start / whole
- `findall` / `finditer` — all matches
- `split` — break on a pattern
- `compile` — reuse, plus `pos` / `endpos`

Python — Substitution & the regex Module

REPLACING, AND WHEN TO REACH FOR REGEX

`re.sub(pattern, repl, string, count=0, flags=0)` returns a new string with matches replaced; `re.subn` also returns how many replacements it made. `repl` is either a **template** string (where `\1`, `\g<name>` and escapes like `\n` are processed) or a **function** that receives each Match and returns the replacement. When the built-in engine runs out of features (Unicode properties, variable-length lookbehind, recursion, fuzzy matching, timeouts), install the third-party **regex** module: it's largely a drop-in replacement.

VERSION NOTES

- 3.7: `re.sub` handles empty matches next to previous matches consistently
- 3.11: atomic groups `(?>…)` and possessive `*+ ++ ?+`
- 3.13: passing `count`, `maxsplit` or `flags` positionally is deprecated
- `pip install regex`, then `import regex as re` for most code

SUBSTITUTION & THE REGEX MODULE

```
import re
re.sub(r"\s+", " ", "a   b \t c")                    # 'a b c'
re.sub(r"(\w+)@(\w+)", r"\2 at \1", "me@site")        # 'site at me'
re.sub(r"(?P<u>\w+)@", r"\g<u> at ", "me@site")       # 'me at site'
re.subn(r"o", "0", "foo boo")                         # ('f00 b00', 4)
re.sub(r"o", "0", "foo", count=1)                     # 'f0o'
re.sub(r"\d+", lambda m: str(int(m[0]) * 2), "3 cats, 10 dogs")  # '6 cats, 20 dogs'
prices = {"apple": "1.20", "pear": "0.90"}
re.sub(r"\{(\w+)\}", lambda m: prices.get(m[1], m[0]), "{apple}/{kiwi}")  # '1.20/{kiwi}'
import regex
regex.findall(r"\p{Lu}\p{Ll}+", "Élan Vital ölçü")    # ['Élan', 'Vital']
regex.search(r"(?<=\$\d+)\.\d\d", "$12.50")[0]        # '.50'
len(regex.findall(r"\X", "éa")), len(re.findall(r".", "éa"))  # (2, 3)
regex.search(r"(?:color){e<=1}", "colr")[0]           # 'colr'
regex.findall(r"\d\d", "1234", overlapped=True)       # ['12', '23', '34']
```

CODE EXPLAINED

- → `\2 at \1` — swap groups in a template
- → `\g<u>` — named group in a template
- → `subn` — returns `(new_string, count)`
- → `lambda m:` — compute each replacement; unknown keys fall back to `m[0]`
- → `regex`: `\p{…}`, any-length lookbehind, `\X` graphemes
- → `{e<=1}` — fuzzy: up to 1 edit; `overlapped=True` — overlapping matches

TEMPLATE ESCAPES

IN REPL	INSERTS
`\1 … \99`	group by number
`\g<1>`	group 1 (safe before a digit)
`\g<name>`	named group
`\g<0>`	whole match
`\n \t`	newline, tab
`\\`	one literal backslash

REGEX MODULE EXTRAS

FEATURE	SYNTAX / OPTION
Unicode properties	`\p{L} \p{Script=Greek}`
variable lookbehind	`(?<=\w+)`
recursion, branch reset	`(?R) (?1) (?|…)`
set operations	`(?V1)[\w--\d]`
fuzzy matching	`(?:word){e<=2}`
overlapping finds	`findall(…, overlapped=True)`
timeouts	`match(…, timeout=0.5)`

⚠ COMMON MISTAKES

- Inserting user text as a template: a `\1` or `\g` in it is interpreted; pass `lambda m: text` instead
- `re.sub(p, r, s, 1)` — positional `count` is deprecated; write `count=1`
- Expecting `re.sub` to modify the string in place; strings are immutable, so assign the result

✓ TIPS & BEST PRACTICES

- Use a dict + function for multi-token templating: `lambda m: table[m[1]]`
- For many literal replacements, one alternation `\b(?:a|b|c)\b` with a dict lookup beats repeated `sub` calls
- `regex` is a near drop-in: try `import regex as re` when `re` lacks a feature

QUICK SUMMARY

- `re.sub(p, repl, s)` — replace all
- `re.subn` — replace and count
- `\g<name>` / function — template / computed
- `regex` — \p{}, lookbehind, fuzzy, timeout

Java — Pattern & Matcher

Pattern.compile · flags · matches() vs find() vs lookingAt() · group(n) / group("name") · start() / end() · while (m.find()) · results()

JAVA.UTIL.REGEX

Java splits regex work between two classes. A **Pattern** is the compiled, immutable regex: build it once with `Pattern.compile(regex, flags)` and share it. A **Matcher** applies that pattern to one input and holds the search state (current position, last match), so you call `find()` in a loop and read `group()`, `start()` and `end()` after each success. Java's flavor is close to PCRE: possessive quantifiers, atomic groups, `\p{…}`, lookbehind with a bounded length, and class intersection &&.

KEY POINTS

- `Pattern` is thread-safe; a `Matcher` is not, so create one per use
- Call `find()` / `matches()` before `group()`, or it throws `IllegalStateException`
- `CASE_INSENSITIVE` is ASCII-only unless you add `UNICODE_CASE`
- Backslashes are doubled in Java strings: `\\d` (sheet 19)

PATTERN & MATCHER

```
import java.util.regex.*;
Pattern p = Pattern.compile("(?<name>\\w+)=(?<val>\\d+)");
Matcher m = p.matcher("a=1, b=22, c=333");
while (m.find()) {
    System.out.println(m.group("name") + "->" + m.group(2) + " @" + m.start());
}                                       // a->1 @0   b->22 @5   c->333 @11
p.matcher("a=1").matches();             // true   (whole input)
p.matcher("a=1; x").matches();          // false
p.matcher("a=1; x").lookingAt();        // true   (prefix)
p.matcher("a=1 b=2").results()          // Java 9+
    .map(r -> r.group(2)).toList();     // [1, 2]
```

CODE EXPLAINED

- → `(?<name>…)` — read with `m.group("name")` or by number
- → `while (m.find())` — the standard loop over all matches
- → `matches()` — entire input; `lookingAt()` — from the start; `find()` — anywhere
- → `results()` — a `Stream<MatchResult>`

PATTERN

METHOD	PURPOSE
`compile(re, flags)`	compile once, reuse
`matcher(input)`	new Matcher
`matches(re, s)`	static one-shot full match
`split(s, limit)`	split; `splitAsStream(s)`
`asMatchPredicate()`	full match (Java 11+)

MATCHER

METHOD	PURPOSE
`find()` `find(i)`	next match / from index i
`matches()`	whole input; `lookingAt()` prefix
`group(n|name)`	matched text
`start()` `end()`	indexes, optionally per group
`results()`	`Stream` of matches (9+)
`reset(s)` `region(a,b)`	reuse / limit the range

FLAGS

CONSTANT	INLINE
`CASE_INSENSITIVE`	`(?i)`
`MULTILINE`	`(?m)`
`DOTALL`	`(?s)`
`COMMENTS`	`(?x)`
`UNICODE_CASE`	`(?u)`
`UNICODE_CHARACTER_CLASS`	`(?U)`

⚠ COMMON MISTAKES

- Reading `m.group()` without a successful `find()` first: `IllegalStateException: No match found`
- Sharing one `Matcher` across threads; its state is mutable, so results interleave
- Expecting `matches()` to search; it requires the **whole** input to match

✓ TIPS & BEST PRACTICES

- Store patterns as `private static final Pattern` fields
- Use `UNICODE_CHARACTER_CLASS` to make `\w`, `\d` and `\b` Unicode-aware
- `p.asPredicate()` plugs straight into `stream.filter(…)`

QUICK SUMMARY

- `Pattern.compile` — immutable, shareable
- `Matcher` — stateful search over one input
- `find / matches / lookingAt` — anywhere / whole / prefix
- `group / start / end` — read the last match

Java — String Methods, Replacement & Gotchas

"\\d+" escaping & text blocks · String.matches() · replaceAll / replaceFirst · split + limit · quoteReplacement · Pattern.quote · Kotlin Regex

REGEX THROUGH JAVA.LANG.STRING

`String` has four regex shortcuts: `matches`, `replaceAll`, `replaceFirst` and `split`. They're convenient but compile the pattern on **every call**, and each has a trap: `matches` needs the **whole** string, `split` treats its argument as a regex and drops trailing empty strings, and the replacement string treats `$` and `\` as special. Java has no raw string literals, so every regex backslash is written twice; text blocks (`"""`) still process escapes and don't change that.

ESCAPING LAYERS

- Regex `\d` → Java `"\\d"`
- Regex `\.` → Java `"\\."`
- Literal backslash: regex `\\` → Java `"\\\\"`
- Replacement `$` → `"\\$"` ; or wrap text in `Matcher.quoteReplacement`

STRING METHODS (JAVA)

```
String s = "a1b22c333";
s.matches("[a-z0-9]+");                      // true   (whole string)
s.matches("\\d+");                           // false
s.replaceAll("\\d+", "#");                   // "a#b#c#"
s.replaceFirst("\\d+", "#");                 // "a#b22c333"
"2026-03-14".replaceAll("(\\d+)-(\\d+)-(\\d+)", "$3/$2/$1");   // "14/03/2026"
"a,b,,c,,".split(",");                       // [a, b, , c]
"a,b,,c,,".split(",", -1);                   // [a, b, , c, , ]
"a.b.c".split(".");                          // []
"a.b.c".split(Pattern.quote("."));           // [a, b, c]
"cost".replaceAll("cost", "\\$5");           // "$5"
"x".replaceAll("x", Matcher.quoteReplacement("$1\\"));   // "$1\\"
Pattern.compile("\\d+").matcher("3 and 4")
    .replaceAll(r -> String.valueOf(Integer.parseInt(r.group()) * 10));  // "30 and 40"
// Kotlin: raw strings need no doubling
Regex("""(\d+)-(\d+)""").replace("10-20", "$2-$1")      // "20-10"
"\\d+".toRegex().findAll("a1b22").map { it.value }.toList()  // [1, 22]
```

CODE EXPLAINED

- → `matches` — implicit `^…$`
- → `split(",")` — trailing empty strings removed; `-1` keeps them
- → `split(".")` — `.` matches everything, so every piece is empty
- → `\\$5` — literal `$` in a replacement
- → `quoteReplacement` — makes any text safe as a replacement
- → `replaceAll(Function)` — computed replacement (Java 9+)

STRING SHORTCUTS

METHOD	BEHAVIOR
`matches(re)`	whole string must match
`replaceAll(re, rep)`	replace every match
`replaceFirst(re, rep)`	replace the first match
`split(re)`	drops trailing empty strings
`split(re, n)`	n>0 max parts; n<0 keep empties
`replace(a, b)`	**literal**, not regex

KOTLIN REGEX

API	PURPOSE
`Regex("""\d+""")`	raw string: single backslashes
`"…".toRegex()`	from a String
`re.containsMatchIn(s)`	find anywhere
`re.matches(s)`	whole string
`re.find(s) / findAll(s)`	`MatchResult?` / sequence
`re.replace(s) { … }`	computed replacement
`m.destructured`	`val (a, b) = …`

⚠ COMMON MISTAKES

- `s.split("|")` or `split(".")` : both are regex metacharacters; use `"\\|"` or `Pattern.quote`
- Calling `String.matches` in a hot loop; it compiles each time, so hoist a `Pattern`
- Using `replaceAll` for a literal swap; `replace` is literal and faster

✓ TIPS & BEST PRACTICES

- Use `Pattern.quote` for literal search text and `Matcher.quoteReplacement` for literal replacements
- Check patterns in regex101's Java flavor, then double every backslash when pasting
- In Kotlin, prefer raw strings `"""…"""` for patterns

QUICK SUMMARY

- `"\\d"` — Java needs doubled backslashes
- `matches` — whole string only
- `split(re, -1)` — keep trailing empties
- `quote` / `quoteReplacement` — literal text

JavaScript — RegExp Basics

/pattern/flags · new RegExp(str, flags) · test() · exec() & lastIndex · match() with / without g · matchAll() · search() · split(/re/)

REGEXP IN JAVASCRIPT

JavaScript has regex built into the language. A **literal** `/\d+/g` is compiled when the script loads; the **constructor** `new RegExp(str, flags)` builds one at runtime from a string, which means backslashes must be doubled (`"\\d+"`) or written with `String.raw`. Patterns are used through RegExp methods (`test`, `exec`) and String methods (`match`, `matchAll`, `search`, `replace`, `split`). A regex with the `g` or `y` flag is **stateful**: it remembers where the last match ended in `lastIndex`.

KEY POINTS

- `match` without `g` returns groups; with `g` it returns only the matched strings
- `matchAll` requires `g` and gives full match objects for every match
- Escape `/` inside a literal: `/https?:\/\//`
- Read `re.source` and `re.flags` to inspect a pattern

REGEXP BASICS

```
const re = /(\d{4})-(\d{2})/;
re.test("Due 2026-10")                      // true
"Due 2026-10".match(re)                     // ['2026-10', '2026', '10']
"a1 b22 c333".match(/\d+/g)                 // ['1', '22', '333']
"none".match(/\d+/g)                        // null
[..."a=1 b=2".matchAll(/(\w)=(\d)/g)].map(m => m[2])  // ['1', '2']
"hello world".search(/o/)                   // 4
"a, b ,c".split(/\s*,\s*/)                  // ['a', 'b', 'c']
"a1b2".split(/(\d)/)                        // ['a', '1', 'b', '2', '']
/(?<y>\d{4})/.exec("in 2026").groups.y      // '2026'
new RegExp("\\d+", "g").source              // '\d+'
new RegExp(String.raw`\d+\.\d+`).test("3.14")  // true
// Stateful g flag:
// const g = /a/g;  g.test("ab") → true;  g.test("ab") → false;  g.lastIndex → 0
```

CODE EXPLAINED

- → `test` — boolean; the fastest check
- → `match(re)` without `g` — array with groups plus `index`
- → `match(/…/g)` — all matched strings, no groups; `null` if none
- → `matchAll` — iterator of full match objects
- → `split` with a capture group keeps the delimiters
- → Second `g.test` starts at `lastIndex` 1, fails, and resets to 0

METHODS

METHOD	RETURNS
`re.test(s)`	`true` / `false`
`re.exec(s)`	match array or `null`; advances `lastIndex` with g/y
`s.match(re)`	no g: like exec; g: string array or `null`
`s.matchAll(re)`	iterator of match arrays (needs g)
`s.search(re)`	index of first match or −1
`s.split(re, limit)`	array; captured groups included
`s.replace / replaceAll`	sheet 21

MATCH ARRAY

PROPERTY	VALUE
`m[0]`	whole match
`m[1] …`	groups (`undefined` if unmatched)
`m.index`	start position
`m.input`	original string
`m.groups`	named groups object
`m.indices`	`[start, end]` pairs (d flag)

FLAGS

FLAG	MEANING
`g`	global: all matches
`i`	case-insensitive
`m`	^ $ per line
`s`	. matches \n
`u / v`	Unicode / sets
`y`	sticky at `lastIndex`
`d`	match indices

⚠ COMMON MISTAKES

- Reusing a global regex with `test()` or `exec()`: `lastIndex` makes alternate calls fail; drop `g` or reset `lastIndex = 0`
- `new RegExp("\\d+")` written as `"\d+"`: the string becomes `d+`, matching letters
- Calling `matchAll` with a non-global regex: it throws a `TypeError`

✓ TIPS & BEST PRACTICES

- Use `String.raw` with `new RegExp` to avoid double escaping
- Hoist literals out of loops; they're compiled once anyway, but stateful `g` regexes shouldn't be shared
- Use `matchAll` + destructuring: `for (const [, k, v] of s.matchAll(re))`

QUICK SUMMARY

- `/re/flags` — literal; `new RegExp()` — runtime
- `test` / `exec` — RegExp methods
- `match` / `matchAll` / `search` / `split` — String methods
- `g` / `y` — stateful via `lastIndex`

JavaScript — Replace & Modern Features

replace / replaceAll · $1 $<name> $& · replacer (match, p1, …, offset, str, groups) · ES2018 · ES2022 d · ES2024 v · ES2025

REPLACING, AND WHAT'S NEW

`str.replace(pattern, replacement)` replaces the **first** match unless the regex has the g flag; `replaceAll` replaces every match and **requires** g when given a regex. The replacement can be a string with $ tokens or a **function** called once per match. JavaScript's regex flavor has grown quickly: lookbehind, named groups and `\p{…}` (ES2018), match indices (ES2022), set operations (ES2024), and `RegExp.escape` plus inline modifiers (ES2025).

GOOD TO KNOW

- `replace("-", "+")` with a **string** pattern replaces only the first occurrence
- In a replacement string, `$$` is a literal dollar sign
- A replacer function's return value is used literally: no `$` processing
- Check runtime support for ES2025 features (Node 24+, current browsers)

REPLACE & MODERN FEATURES

```
"2026-03-14".replace(/(\d+)-(\d+)-(\d+)/, "$3/$2/$1")    // '14/03/2026'
"a-b-c".replace("-", "+")                                 // 'a+b-c'
"a-b-c".replaceAll("-", "+")                              // 'a+b+c'
"a-b-c".replace(/-/g, "+")                                // 'a+b+c'
"x".replaceAll(/x/, "y")                                  // TypeError: needs g flag
"John Smith".replace(/(?<f>\w+) (?<l>\w+)/, "$<l>, $<f>")  // 'Smith, John'
"hi there".replace(/\b\w/g, c => c.toUpperCase())         // 'Hi There'
"3 + 4".replace(/(\d+) \+ (\d+)/, (m, a, b) => +a + +b)  // '7'
"$5".replace(/\$(\d)/, "$$$1.00")                         // '$5.00'
/(?<=\$)\d+/.exec("cost $42")[0]                          // '42'
/a.c/s.test("a\nc")                                       // true
/(?<y>\d{4})/d.exec("in 2026").indices.groups.y           // [3, 7]
/[\p{L}--[a-z]]/v.test("é")                               // true
// ES2025: RegExp.escape("1+1")  → '\x31\+1' (safe in any position)
// ES2025: /(?i:hello) world/.test("HELLO world")  → true
```

CODE EXPLAINED

- → `$3/$2/$1` — reorder groups
- → String pattern → first only; `replaceAll` or `/g` → all
- → `$<name>` — named group in a replacement
- → `(m, a, b) => …` — replacer gets match, then groups
- → `$$` — literal `$`
- → `d` flag — `[start, end]` for each group
- → `v` flag — set subtraction inside a class

REPLACEMENT TOKENS

TOKEN	INSERTS
`$1 … $99`	numbered group
`$<name>`	named group
`$&`	whole match
`$``	text before the match
`$'`	text after the match
`$$`	literal $

REPLACER FUNCTION ARGUMENTS

ARGUMENT	VALUE
`match`	whole match
`p1, p2, …`	one per group
`offset`	match position
`string`	whole input
`groups`	named groups (if any)

FEATURE TIMELINE

EDITION	ADDED
ES2015	u and y flags
ES2018	s flag, lookbehind, named groups, `\p{…}`
ES2020	`matchAll`
ES2021	`replaceAll`
ES2022	d flag (`.indices`)
ES2024	v flag (set ops, string props)
ES2025	`RegExp.escape`, `(?i:…)`, duplicate names

⚠ COMMON MISTAKES

- Expecting `str.replace("a", "b")` to replace every `a`; use `replaceAll` or `/a/g`
- Inserting user text as a replacement string: its `$&` or `$1` gets expanded; use `() => text`
- Forgetting the replacer's trailing `offset` and `string` arguments when using `...args`

✓ TIPS & BEST PRACTICES

- Use named groups for readable replacements: `$<year>`
- Prefer the function form for anything conditional or computed
- Until `RegExp.escape` is available everywhere, use `s.replace(/[.*+?^${}()|[\]\\]/g, "\\$&")`

QUICK SUMMARY

- `replace` — first match (all with `g`)
- `replaceAll` — every match; regex needs `g`
- `$1 $<name> $& $$` — replacement tokens
- function — computed replacement

Other Languages — Go, C#, Ruby & More

Go regexp (RE2) · C# Regex / [GeneratedRegex] · Ruby =~ / scan / gsub · C++ std::regex · C regcomp · Rust regex · PHP preg_* · Perl

SAME PATTERNS, DIFFERENT HOSTS

Each block shows the same five moves: compile, test, find, extract a group, and replace. The pattern syntax carries over from sheets 01–15; what changes is the **API**, the **string escaping**, the **replacement tokens**, and which **features** the engine supports. Go and Rust use linear-time engines without lookaround or backreferences; the rest are backtracking engines close to PCRE.

BIGGEST DIFFERENCES

- Go/Rust: no lookaround or backrefs; Go `$1x` means group `1x`, so write `${1}x`
- Ruby: `^` `$` match at **every** line; validate with `\A…\z`
- PHP: patterns need delimiters `/…/`; add `u` for UTF-8

GO — REGEXP (RE2 SYNTAX)

```
re := regexp.MustCompile(`(\w+)@(\w+)\.com`)
re.MatchString("me@site.com")              // true
re.FindAllString("a@b.com c@d.com", -1)    // [a@b.com c@d.com]
re.FindStringSubmatch("me@site.com")[2]    // "site"
re.ReplaceAllString("me@site.com", "${2}:$1")  // "site:me"
regexp.Compile(`(?<=a)b`)                  // error: no lookbehind
```

C# — SYSTEM.TEXT.REGULAREXPRESSIONS

```
var re = new Regex(@"(?<user>\w+)@(\w+)", RegexOptions.IgnoreCase);
re.IsMatch("ME@site")                      // true
re.Match("me@site").Groups["user"].Value   // "me"
foreach (Match m in re.Matches(text)) { … }
Regex.Replace("me@site", @"(\w+)@", "$1 at ")  // "me at site"
[GeneratedRegex(@"\d+")] static partial Regex Digits();  // .NET 7+
```

RUBY — BUILT-IN REGEXP

```
"me@site.com" =~ /@(\w+)/            # 2  ($1 == "site")
"me@site.com".match?(/@/)            # true, sets no globals
"a1 b22".scan(/\d+/)                 # ["1", "22"]
"a1 b22".gsub(/\d+/) { |d| d.to_i * 2 }  # "a2 b44"
/(?<user>\w+)@/ =~ "me@x.io"; user   # "me" (named → local var)
"ok\nbad" =~ /\A\w+\z/               # nil (/^\w+$/ would match)
```

RUST — REGEX CRATE

```
let re = Regex::new(r"(?<user>\w+)@(\w+)").unwrap();
re.is_match("me@site")                       // true
&re.captures("me@site").unwrap()["user"]     // "me"
re.find_iter("a@b c@d").map(|m| m.as_str())  // "a@b", "c@d"
re.replace_all("me@site", "$2:$user")        // "site:me"
Regex::new(r"(?<=a)b").is_err()              // true
```

PHP — PCRE (PREG_*)

```
preg_match('/(\w+)@(\w+)/', 'me@site', $m);   // 1; $m[2]: 'site'
preg_match_all('/\d+/', 'a1 b22', $m);        // 2; $m[0]: ['1','22']
preg_replace('/(\w+)@/', '$1 at ', 'me@site') // 'me at site'
preg_replace_callback('/\d+/', fn($m) => $m[0]*2, 'a1')  // 'a2'
preg_split('/\s*,\s*/', 'a, b ,c')            // ['a','b','c']
preg_match('/^\p{Lu}/u', 'Élan')              // 1  (u = UTF-8)
```

PERL — THE ORIGINAL

```
if ("me@site" =~ /(\w+)@(\w+)/) { print $2 }   # site
my @n = ("a1 b22" =~ /\d+/g);                  # (1, 22)
(my $t = "a-b-c") =~ s/-/+/g;                  # a+b+c
my $u = $s =~ s/x/y/gr;                        # /r returns a copy
$s =~ s/(\w+)/\u$1/g;                          # Capitalize Words
my $re = qr/\d{3}/;                            # compiled pattern
```

C++ — <REGEX>

```
std::regex re(R"((\w+)@(\w+))");      // raw string R"(...)"
std::smatch m;
std::regex_search(s, m, re);          // true; m[2] == "site"
std::regex_match(s, re);              // whole string
std::regex_replace(s, re, "$2:$1");   // "site:me"
// for (std::sregex_iterator it(b, e, re), end; it != end; ++it)
```

C — POSIX <REGEX.H>

```
regex_t re;  regmatch_t m[3];
regcomp(&re, "([a-z]+)@([a-z]+)", REG_EXTENDED);
if (regexec(&re, s, 3, m, 0) == 0)          /* 0 = match */
    printf("%.*s\n", (int)(m[2].rm_eo - m[2].rm_so),
           s + m[2].rm_so);                 /* site */
regfree(&re);              /* ERE: no \d, no lazy */
```

⚠ COMMON MISTAKES

- Porting a JS or Python pattern with lookbehind to Go or Rust: it fails to compile
- Validating with `/^…$/` in Ruby; multi-line input slips through
- Using `std::regex` in a hot path in C++; it's often 10× slower than RE2, PCRE2 or CTRE

✓ TIPS & BEST PRACTICES

- Go: use backticks for patterns and `${1}` in replacements
- C#: `[GeneratedRegex]` compiles at build time; `RegexOptions.NonBacktracking` guarantees linear time
- Ruby: prefer `match?` for yes/no checks; it skips setting `$~`

QUICK SUMMARY

- Go / Rust — RE2-style, linear, no lookaround
- C# / Ruby / PHP / Perl — PCRE-like
- C++ — ECMAScript grammar, slow
- C — POSIX ERE via `regcomp`

Command-Line Tools — grep, sed, awk & rg

REGEX ON THE COMMAND LINE

Unix tools speak **POSIX** regex in two dialects. **BRE** (basic, the default for `grep` and `sed`) treats `+ ? | ( ) { }` as literals unless you escape them; **ERE** (extended, `grep -E`, `sed -E`, `awk`) makes them special without backslashes. Neither has `\d` or lazy quantifiers: use `[0-9]` and `[[:space:]]`. For Perl-style syntax use `grep -P` (GNU) or **ripgrep** (`rg`), which is fast, recursive and respects `.gitignore` by default.

GNU VS BSD (MACOS)

- In-place edit: GNU `sed -i 's/…/'`, macOS `sed -i '' 's/…/'`
- macOS `grep` has no `-P`; use `rg -P` or Homebrew's `ggrep`
- `\s`, `\w`, `\b`, `\<` are GNU extensions; `[[:space:]]` works everywhere
- Quote patterns in **single** quotes so the shell leaves `$` and `\` alone

COMMON COMMANDS

```
grep -E '^[0-9]{3}-[0-9]{4}$' phones.txt        # ERE: {} unescaped
grep 'ab\+c\|xy' notes.txt                       # BRE (GNU): escape + and |
grep -oP '(?<=id=)\d+' app.log                   # PCRE: lookbehind, \d (GNU)
grep -rnw 'TODO' src/                            # recursive, line numbers, whole word
grep -ciE 'error|fail' app.log                   # count, case-insensitive
grep -v '^#' config.ini                          # invert: drop comment lines
sed -E 's/([0-9]+)-([0-9]+)/\2-\1/g' file.txt    # swap with backrefs
sed -E 's/[[:space:]]+$//' file.txt              # strip trailing whitespace
sed -n '/BEGIN/,/END/p' file.txt                 # print a range of lines
awk '/ERROR/ {n++} END {print n}' app.log        # count matching lines
awk -F, '$3 ~ /^[A-Z][A-Z]$/ {print $1}' data.csv  # match one field
rg -n '\bfn \w+\(' -t rust                       # ripgrep (Rust regex)
rg -P '(?<=v)\d+\.\d+' CHANGELOG.md              # ripgrep with PCRE2
find . -regextype posix-extended -regex '.*\.(jpg|png)'   # GNU find
```

CODE EXPLAINED

- → `-E` ERE · `-P` PCRE · `-F` fixed string (no regex)
- → `-o` print only the match · `-w` whole word · `-x` whole line
- → `sed -E` + `\1` — backrefs in the replacement; `&` = whole match
- → `sed -n '/a/,/b/p'` — address ranges use regex too
- → `awk` — `/re/` tests the line, `$3 ~ /re/` tests a field
- → `find -regex` matches the **whole path**, not just the name

BRE VS ERE

FEATURE	BRE (GREP, SED)	ERE (-E, AWK)
group	`\( \)`	`( )`
alternation	`|` (GNU)	`|`
1 or more / optional	`\+ \?` (GNU)	`+ ?`
interval	`\{2,3\}`	`{2,3}`
backreference	`\1`	`\1` (GNU)
literal (+ ?	`( + ?`	`\( \+ \?`

EDITORS

SYNTAX	MEANING
`vim \v`	very magic: ERE-like, no escaping
`vim \{-}`	lazy ∗: `.\{-}`
`vim \zs \ze`	set match start / end
`vim :%s/a/b/gc`	replace all, confirm each
VS Code	JS flavor; $1 in replace

⚠ COMMON MISTAKES

- Using `\d` with `grep -E` or `sed -E`: POSIX has no `\d`, so it silently matches the letter `d` (or warns)
- Double-quoting patterns: the shell expands `$var` and `\` before grep sees them
- Running GNU `sed -i` syntax on macOS: `-i` eats the next argument as a backup suffix

✓ TIPS & BEST PRACTICES

- Use `grep -F` for literal strings; no escaping and faster
- Prefer `rg` for code search: recursive, Unicode-aware, skips ignored files
- Test a `sed` edit without `-i` first, then add `-i.bak` to keep a backup

QUICK SUMMARY

- `BRE` — escape `\( \) \{ \} \+ \?`
- `ERE` — `grep -E`, `sed -E`, `awk`
- `grep -P` / `rg -P` — Perl syntax
- `[[:space:]]` — portable classes

Quick Reference — Validation Patterns

email · URL · IPv4 / IPv6 · UUID / GUID · ISO 8601 dates & times · phone (E.164 / NANP) & ZIP · hex color & semver · password lookaheads

COPY-READY VALIDATION PATTERNS

Each pattern is anchored with `^…$` for whole-string validation and written PCRE-style: use `\A…\z` in Ruby and double the backslashes in Java strings. They're **practical**, not RFC-complete. **Shape isn't validity**: the date pattern accepts `2026-02-30`; check values in code. Patterns tagged **✗ Go** use lookaround, which Go and Rust don't support.

INTERNET & IDENTIFIERS

VALIDATES	PATTERN	MATCHES	NOTES
Email (pragmatic)	`^[\w.%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$`	`jo.lee+tag@mail.example.com`	no bare hosts
URL (http/https)	`^https?://[^\s/$.?#][^\s]*$`	`https://ex.com/a?b=1`	shape only
Domain name	`^(?=.{1,253}$)(?:(?!-)[A-Za-z0-9-]{1,63}(?<!-)\.)+[A-Za-z]{2,63}$`	`sub.example.co.uk`	✗ Go
IPv4	`^(?:(?:25[0-5]|2[0-4]\d|1\d\d|[1-9]?\d)\.){3}(?:25[0-5]|2[0-4]\d|1\d\d|[1-9]?\d)$`	`192.168.0.1`	0–255 each
IPv6 (full form)	`^(?:[0-9A-Fa-f]{1,4}:){7}[0-9A-Fa-f]{1,4}$`	`2001:db8:0:0:0:ff00:42:8329`	no `::` form
MAC address	`^(?:[0-9A-Fa-f]{2}[:-]){5}[0-9A-Fa-f]{2}$`	`00:1A:2b:3C:4d:5E`	
UUID / GUID	`^[0-9a-f]{8}-[0-9a-f]{4}-[1-8][0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$`	`123e4567-e89b-42d3-a456-4266…`	v1–8; `i` flag
Base64	`^(?:[A-Za-z0-9+/]{4})*(?:[A-Za-z0-9+/]{2}==|[A-Za-z0-9+/]{3}=)?$`	`aGVsbG8=`	URL-safe: `-_`
JWT (shape)	`^[\w-]+\.[\w-]+\.[\w-]*$`	`eyJhbGc.eyJzdWI.SflKxw`	shape only

DATES, CONTACT & FORMATS

VALIDATES	PATTERN	MATCHES	NOTES
ISO 8601 date	`^\d{4}-(?:0[1-9]|1[0-2])-(?:0[1-9]|[12]\d|3[01])$`	`2026-10-10`	accepts `02-30`
ISO 8601 date-time	`^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}(?::\d{2}(?:\.\d+)?)?(?:Z|[+-]\d{2}:\d{2})$`	`2026-10-10T14:05:00Z`	offset required
Time, 24-hour	`^(?:[01]\d|2[0-3]):[0-5]\d(?::[0-5]\d)?$`	`23:59`, `07:05:09`	
Time, 12-hour	`^(?:0?[1-9]|1[0-2]):[0-5]\d\s?[AaPp][Mm]$`	`9:30 pm`, `12:00AM`	
US date	`^(?:0?[1-9]|1[0-2])/(?:0?[1-9]|[12]\d|3[01])/\d{4}$`	`10/10/2026`	M/D/YYYY
Phone, E.164	`^\+[1-9]\d{1,14}$`	`+14155550123`	canonical form
Phone, US / NANP	`^(?:\+?1[-. ]?)?\(?([2-9]\d{2})\)?[-. ]?([2-9]\d{2})[-. ]?(\d{4})$`	`(415) 555-0123`	3 groups
US ZIP / ZIP+4	`^\d{5}(?:-\d{4})?$`	`53202-1234`	
Canada postal	`^[A-Za-z]\d[A-Za-z][ -]?\d[A-Za-z]\d$`	`K1A 0B1`	
UK postcode	`^[A-Z]{1,2}\d[A-Z\d]? ?\d[A-Z]{2}$`	`SW1A 1AA`	simplified; add `i`
Hex color	`^#(?:[0-9a-fA-F]{3,4}){1,2}$`	`#fff`, `#1a2b3c80`	3, 4, 6 or 8 digits
Semantic version	`^\d+\.\d+\.\d+(?:-[0-9A-Za-z.-]+)?(?:\+[0-9A-Za-z.-]+)?$`	`2.0.0-rc.1+build.5`	simplified
Password rules	`^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\w\s]).{12,}$`	`Correct-Horse9`	12+ chars; ✗ Go
Username	`^[A-Za-z][\w-]{2,15}$`	`bob_42`	3–16, letter first
URL slug	`^[a-z0-9]+(?:-[a-z0-9]+)*$`	`my-post-2`	
Card number (shape)	`^(?:\d[ -]?){12,18}\d$`	`4111 1111 1111 1111`	+ Luhn check

⚠ COMMON MISTAKES

- No anchors: `\d{5}` accepts "123456789"
- Over-strict rules reject real input (`+` in emails)
- Checking IPs/URLs by regex alone; parse them too

✓ TIPS & BEST PRACTICES

- Normalize first: trim, uppercase, strip punctuation
- One named constant per pattern, with test samples
- Parsers: `ipaddress` , `new URL()` , `URI`

QUICK SUMMARY

- `^…$` — validate the whole value
- regex for shape, code for meaning
- ✗ Go — needs lookaround

Sheet 25

Quick Reference — Text Processing & Extraction

trim / collapse whitespace · blank & duplicate lines · repeated words \b(\w+)\s+\1\b · quoted strings · numbers · key=value & logs · comments · camelCase ↔ snake_case

COPY-READY TEXT PATTERNS

Search-and-replace and extraction patterns for everyday text work, PCRE-style and unanchored. **Flags** column: `m` = multiline, `i` = case-insensitive; write `\1` as `$1` in JS, Java and .NET. Patterns that touch code or markup (comments, tags, quoted strings) are **naive**: they don't know about context like a `//` inside a string, so check results before bulk edits.

CLEAN UP (FIND → REPLACE)

TASK	FIND	REPLACE	FLAGS	EFFECT
Trim both ends	`^\s+|\s+$`	(empty)		`" hi "` → `"hi"`
Collapse spaces / tabs	`[ \t]{2,}`	a space		`a  b` → `a b`
Trailing whitespace	`[ \t]+$`	(empty)	m	per line
Remove blank lines	`^[ \t]*\n`	(empty)	m	incl. whitespace-only
Duplicate lines	`^(.*)(?:\n\1)+$`	`\1`	m	consecutive repeats
Repeated words	`\b(\w+)\s+\1\b`	`\1`	i	`the the` → `the`

EXTRACT

FINDS	PATTERN	EXAMPLE MATCH
Double-quoted string	`"(?:[^"\\]|\\.)*"`	`"a \"q\" b"`; same idea for `'…'`
Line comment	`//.*$ or #.*$`	`// set a` (with `m`)
Block comment	`/\*[\s\S]*?\*/`	`/* two\nlines */`
HTML / XML tag	`</?[A-Za-z][^>]*>`	`<p class="x">`, `</b>`
Integer	`[-+]?\b\d+\b`	`-12`, `+7`, `30`
Decimal / scientific	`[-+]?(?:\d+(?:\.\d*)?|\.\d+)(?:[eE][-+]?\d+)?`	`3.14`, `.25`, `6.02e23`
Thousands separators	`\b\d{1,3}(?:,\d{3})+(?:\.\d+)?\b`	`1,234,567.89`
Currency (USD)	`\$\d{1,3}(?:,\d{3})*(?:\.\d{2})?`	`$1,299.00`, `$5`
key=value	`(\w+)\s*=\s*("[^"]*"|\S+)`	`host = "a b"`, `port=80`
URL in text	`https?://[^\s<>"']+`	`https://ex.com/a?b=1`
Email in text	`[\w.+-]+@[\w-]+(?:\.[\w-]+)+`	`jo@ex.co.uk`
@mention / #tag	`(?<!\w)[#@]\w+`	`@ann`, `#regex` (not `a#b`)
File extension	`\.([A-Za-z0-9]+)$`	`report.final.PDF` → `PDF`
Query parameter	`[?&]id=([^&#]*)`	`/p?x=1&id=42#top` → `42`
CSV field	`(?:^|,)("(?:[^"]|"")*"|[^,]*)`	`"say ""hi""", "b,c"`
Apache access log	`^(\S+) \S+ (\S+) \[([^\]]+)\] "(\S+) (\S+) [^"]*" (\d{3}) (\d+|-)`	ip, user, time, method, path, status, bytes

CASE CONVERSION

TASK	FIND → REPLACE
camelCase → snake_case	`(?<=[a-z0-9])([A-Z])` → `_\1`, then lowercase
snake_case → camelCase	`_([a-z])` → function: upper of group 1
Split camelCase	`split((?<=[a-z])(?=[A-Z]))` → `parse Http`
Capitalize words	`\b\w` → function: upper

DON'T USE REGEX FOR THIS

DATA	USE INSTEAD
HTML / XML	a parser: BeautifulSoup, `DOMParser`, lxml
JSON	`json.loads`, `JSON.parse`
CSV with quotes	`csv` module or a CSV library
Dates & times	`datetime.strptime`, `Date` / Temporal
URLs	`urllib.parse`, `new URL()`

⚠ COMMON MISTAKES

- Naive comment/tag patterns break on strings and nesting
- Forgetting `m`: `^` and `$` then match only once

✓ TIPS & BEST PRACTICES

- Preview matches with `rg` before a bulk replace
- Use a replacement function for computed output

QUICK SUMMARY

- `\s` / `[ \t]` — whitespace cleanup
- `(?:…)` + specific classes — extraction
- parser — for nested formats